# Cloud Computing

From Basic Concepts to Deployment Models

September 21, 2026

# Contents


**1 Cloud Computing** **1**

1.1 Let's Start With Two Simple Questions . . . 1
1.2 What Is Cloud Computing? . . . 1
1.3 First, Understand the Word "Cloud" . . . 2
1.4 Real-Life Example 1 — Google Drive . . . 2
1.5 Real-Life Example 2 — Gmail . . . 2
1.6 Why Do We Need Cloud Computing? . . . 3
1.7 Advantages of Cloud Computing . . . 3
1.8 Disadvantages of Cloud Computing . . . 4
1.9 Popular Cloud Services We Use Every Day . . . 4
1.10 Recent Trends in Cloud Computing . . . 4
1.10.1 Trends 1–5 . . . 4
1.10.2 Trends 6–10 . . . 5
1.11 The Evolution of Cloud Computing . . . 5
1.11.1 A 70-Year Journey . . . 5
1.11.2 Mainframe Computing (1950 – 1970) . . . 6
1.11.3 Distributed Systems (1970 – 1980) . . . 6
1.11.4 Cluster Computing (1980 – 1990) . . . 6
1.11.5 Grid Computing (1990 – 2000) . . . 7
1.11.6 Utility Computing (Late 1990s – 2000s) . . . 7
1.11.7 Virtualization (1980 – Present) . . . 8
1.11.8 Web 2.0 (Gained Popularity ∼2004) . . . 8
1.11.9 Service Orientation (2000s) . . . 8
1.11.10 All Roads Lead to the Cloud . . . 9
1.12 Public vs Private vs Hybrid Cloud . . . 9
1.12.1 Three Deployment Models, One Decision . . . 9
1.12.2 Public Cloud — Owned & Shared by a Provider . . . 10
1.12.3 Private Cloud — Dedicated to One Organization . . . 10
1.12.4 Hybrid Cloud — The Best of Both Worlds . . . 11
1.12.5 Head-to-Head Comparison . . . 11
1.12.6 Which One Should You Choose? . . . 12
1.12.7 Quick Revision Summary . . . 12
1.13 Categories of Cloud Computing: Everything as a Service . . . 13
1.13.1 Infrastructure as a Service (IaaS) . . . 13
1.13.1.1 What Does IaaS Provide? . . . 13
1.13.1.2 Examples of IaaS . . . 14
1.13.1.3 Advantages of IaaS . . . 14

1.13.1.4 Simple Example . . . 14
1.13.2 Platform as a Service (PaaS) . . . 14
1.13.2.1 What Does PaaS Provide? . . . 14
1.13.2.2 Examples of PaaS . . . 15
1.13.2.3 Advantages of PaaS . . . 15
1.13.2.4 Simple Example . . . 15
1.13.3 Software as a Service (SaaS) . . . 15
1.13.3.1 Examples of SaaS . . . 16
1.13.3.2 Advantages of SaaS . . . 16
1.13.3.3 Simple Example . . . 16
1.13.4 Security as a Service (SECaaS) . . . 16
1.13.4.1 What Can SECaaS Provide? . . . 17
1.13.4.2 Advantages of SECaaS . . . 17
1.13.4.3 Simple Example . . . 17
1.14 Understanding the Difference Between IaaS, PaaS and SaaS . . . 18
1.15 An Easy Way to Remember IaaS, PaaS and SaaS . . . 19
1.16 Quick Revision: Cloud Service Models . . . 19
1.17 Pros and Cons of Cloud Computing . . . 20
1.17.1 Advantages of Cloud Computing . . . 20
1.17.1.1 Cost Reduction . . . 20
1.17.1.2 Scalability . . . 20
1.17.1.3 Accessibility . . . 20
1.17.1.4 Flexibility . . . 21
1.17.1.5 High Availability . . . 21
1.17.1.6 Backup and Disaster Recovery . . . 21
1.17.1.7 Automatic Updates . . . 21
1.17.1.8 Faster Deployment . . . 21
1.17.2 Disadvantages of Cloud Computing . . . 21
1.17.2.1 Internet Dependency . . . 21
1.17.2.2 Security and Privacy Risks . . . 22
1.17.2.3 Vendor Lock-In . . . 22
1.17.2.4 Limited Control . . . 22
1.17.2.5 Downtime . . . 22
1.17.2.6 Unpredictable Costs . . . 23
1.17.2.7 Performance Issues . . . 23
1.17.2.8 Compliance and Legal Issues . . . 23
1.17.3 Advantages and Disadvantages at a Glance . . . 23
1.17.4 Simple Example . . . 23
1.17.4.1 Traditional Approach . . . 24
1.17.4.2 Cloud Approach . . . 24
1.18 What Is Virtualization? . . . 24
1.19 Levels of Virtualization Implementation . . . 25
1.19.1 Instruction Set Architecture (ISA) Level . . . 25
1.19.2 Hardware Abstraction Level (HAL) . . . 26
1.19.3 Operating System Level . . . 26
1.19.4 Library Support Level . . . 26
1.19.5 User-Application Level . . . 27
1.19.6 Relative Merits of Different Virtualization Levels . . . 27

1.20 VMM Design Requirements and Providers . . . 27
1.21 Virtualization Support at the OS Level . . . 29
1.21.1 Why OS-Level Virtualization? . . . 29
1.21.2 Advantages of OS Extensions . . . 29
1.21.3 Disadvantages of OS Extensions . . . 30
1.21.4 Virtualization on Linux and Windows Platforms . . . 31
1.22 Middleware Support for Virtualization . . . 32
1.23 Virtualization Structures, Tools, and Mechanisms . . . 33
1.23.1 Hypervisor and Xen Architecture . . . 33
1.23.2 Para-Virtualization with Compiler Support . . . 34
1.24 Virtualization of CPU, Memory, and I/O Devices . . . 34
1.24.1 Hardware Support for Virtualization . . . 34
1.24.2 CPU Virtualization . . . 35
1.24.3 Memory Virtualization . . . 36
1.24.4 I/O Virtualization . . . 37
1.24.5 Virtualization in Multi-Core Processors . . . 41
1.25 Desktop Virtualization . . . 41
1.25.1 What Is Desktop Virtualization? . . . 41
1.25.2 Virtual Desktop Infrastructure (VDI) . . . 42
1.25.3 VDI Access Flow: Local vs. Remote Users . . . 42
1.25.4 VDI in the Real World . . . 42
1.25.5 Popular Virtualization / VDI Software . . . 44
1.26 Virtualization for Data Center Automation: The Big Picture . . . 45
1.26.1 Start With a Simple Problem . . . 45
1.26.2 What Does “Automation” Mean Here? . . . 45
1.26.3 Our Roadmap . . . 46
1.27 Virtual Clusters . . . 46
1.27.1 First, What Is a Cluster? . . . 46
1.27.1.1 Why Do We Use Clusters? . . . 47
1.27.2 What Is a Physical Cluster? . . . 48
1.27.3 Why Do We Need Virtual Clusters? . . . 48
1.27.4 Definition of a Virtual Cluster . . . 49
1.27.5 Physical Cluster vs. Virtual Cluster . . . 50
1.27.6 Virtual Cluster Architecture . . . 50
1.27.7 How a Virtual Cluster Is Created (Dynamic Provisioning) . . . 51
1.27.8 Real-Life Example: An Online Shopping Website . . . 52
1.27.9 Template VMs: Make One, Copy Many . . . 52
1.27.10 VM Image Management . . . 53
1.28 Resource Management in the Cloud . . . 53
1.28.1 What Is Resource Management? . . . 53
1.28.2 Why Is It Needed? . . . 54
1.28.3 Resource Management Architecture . . . 55
1.28.4 Main Functions of Resource Management . . . 55
1.28.5 Worked Example 1: Resource Allocation . . . 56
1.28.6 Worked Example 2: Resource Scheduling . . . 56
1.28.7 Load Balancing . . . 57
1.28.8 Server Consolidation . . . 58
1.29 VM Migration . . . 58

1.29.1 What Is VM Migration? . . . 58
1.29.2 Why Do We Migrate VMs? . . . 58
1.29.3 Live VM Migration . . . 59
1.29.4 Live Migration Steps . . . 59
1.29.5 Worked Example of Live Migration . . . 59
1.29.6 Migration Performance Factors . . . 60
1.29.7 Memory Migration . . . 60
1.29.7.1 Iterative Memory Transfer (Pre-Copy) . . . 61
1.29.8 File System Migration . . . 61
1.29.9 Network Migration . . . 62
1.30 Dynamic Resource Allocation and Elasticity . . . 62
1.30.1 Dynamic Resource Allocation . . . 62
1.30.2 The Resource Management Decision Cycle . . . 62
1.30.3 Example: A University Cloud . . . 63
1.30.4 Static vs. Dynamic Allocation . . . 64
1.30.5 Summary of Resource Management Techniques . . . 64
1.30.6 Fault Tolerance . . . 64
1.31 Virtual Cluster Management Models . . . 65
1.31.1 Guest-Based vs. Host-Based Management . . . 65
1.32 Advantages and Challenges of Virtual Clusters . . . 66
1.33 Putting It All Together . . . 66
1.33.1 Complete Resource Management Flow . . . 66
1.33.2 End-to-End Example: A Machine-Learning Experiment . . . 67
1.33.3 Another Example: An E-Commerce Company . . . 67
1.34 Virtualization for Data-Center Automation . . . 67
1.34.1 What Is Data-Center Automation? . . . 67
1.34.1.1 Market Trend (A Little History) . . . 68
1.34.1.2 The Four Topics We Will Study . . . 68
1.34.2 Server Consolidation in Data Centers . . . 68
1.34.2.1 The Problem: Two Kinds of Workloads . . . 69
1.34.2.2 The Solution: Server Consolidation . . . 69
1.34.2.3 Benefits of Virtualization-Based Consolidation . . . 70
1.34.2.4 What Must Be Considered to Automate a Data Center? 70
1.34.2.5 Scheduling at Three Levels . . . 70
1.34.2.6 Dynamic CPU Allocation and the Two-Level Controller 71
1.34.2.7 Multicore Processors and Power Budgeting . . . 71
1.34.3 Virtual Storage Management . . . 71
1.34.3.1 What Does “Storage Virtualization” Mean Here? . . . 71
1.34.3.2 Encapsulation and Isolation . . . 72
1.34.3.3 Why Is Storage So Difficult to Manage? . . . 72
1.34.3.4 Two Ideas: CAS and Parallax . . . 72
1.34.3.5 Example: How Parallax Works . . . 73
1.34.4 Cloud OS for Virtualized Data Centers . . . 73
1.34.4.1 What Is a Cloud OS? . . . 73
1.34.4.2 Four Well-Known VI Managers . . . 74
1.34.4.3 Example 1: Eucalyptus . . . 74
1.34.4.4 Example 2: VMware vSphere 4 (a Commercial Cloud OS) 75
1.34.5 Trust Management in Virtualized Data Centers . . . 76

1.34.5.1 Why Does Virtualization Bring New Security Questions? 76
1.34.5.2 A Subtle Problem: Rolling Back a VM . . . . . 77
1.34.5.3 VM-Based Intrusion Detection . . . . . 77
1.34.5.4 Honeypots and Honeynets . . . . . 78
1.34.5.5 Example: Trusted Zones for Virtual Clusters (EMC and VMware) . . . . . 78
1.34.6 Putting the Four Topics Together . . . . . 79
1.34.7 Likely Exam Questions and Short Answers . . . . . 79
1.34.8 Quick Revision Summary . . . . . 80
1.34.9 The Big Question: Where Does Your Request Run? . . . . . 80
1.35 Practice Section . . . . . 81
1.35.1 Group Problem: Where Should the VM Go? . . . . . 81
1.35.2 Check Your Understanding . . . . . 81
1.35.3 Exam Preparation . . . . . 82

# Chapter 1

# Cloud Computing

## 1.1 Let's Start With Two Simple Questions

Think about this for a second:

1. How many of us use Google Drive, Gmail, YouTube, or WhatsApp every day?
2. Where do you think your photos, emails, and videos are actually stored? Inside your phone?

**The surprising answer:** No, most of that data is *not* stored inside your phone. It is stored in huge, powerful computers called **servers**, which sit somewhere else in the world — maybe thousands of kilometers away. You simply access that data through the **Internet**.

This whole idea — storing and using data or software that lives on someone else's computer, accessed over the internet — is called **Cloud Computing**.

## 1.2 What Is Cloud Computing?

**Core Concept**

Cloud Computing means using computing services — like **storage, software, databases, and servers** — through the Internet, instead of relying on your own personal computer.

**Even simpler way to think about it:** Instead of buying and maintaining an expensive, powerful computer yourself, you **rent** computing resources whenever you need them, and you access them through the internet — just like renting a car instead of buying one.

The four main things you can "rent" from the cloud are:

- **Storage** – space to keep your files
- **Software** – applications you can use online
- **Servers** – powerful machines that do the processing

- **Databases** – organized places to store and manage data

## 1.3 First, Understand the Word "Cloud"

The word "cloud" is not magic — it simply means:

> A collection of powerful computers (servers) connected together via the Internet.

These servers don't sit in a random room — they live inside massive buildings called **Data Centers**. Key facts about data centers:

- They contain **thousands of servers**
- They are **powered 24 hours a day, 7 days a week** (they never really switch off)
- They are **heavily guarded and secured** (physically and digitally)
- They are **connected to the world via the Internet**

So basically, when people say "it's in the cloud," they really mean "it's saved on someone else's server, in some data center, and you can reach it over the internet."

## 1.4 Real-Life Example 1 — Google Drive

**The scenario:** Imagine your phone has:

- 5 GB of photos
- 10 GB of videos
- But only 2 GB of free storage space left

**What do you do?** Instead of deleting your precious files, you upload them to **Google Drive**.

**What actually happens behind the scenes:**

1. Your phone sends the files over the **Internet**
2. The files travel to a **Google Data Center**
3. Your files are now stored there — NOT inside your phone anymore

The best part? You can download or view these files anytime, from any device, as long as you have internet access.

## 1.5 Real-Life Example 2 — Gmail

**Question:** When you send an email to a friend, where does it actually get stored?

**Answer:** Not on your laptop. It gets stored on **Google's cloud servers**.

**Why this matters:** Even if your laptop breaks or gets stolen tomorrow, you can simply log into Gmail from any other computer or phone — and your email will still be there,

safe and sound. That's because your email was never really "on" your laptop; it always lived in the cloud.

**Core Concept**

Your data outlives your device — because your data was never stored on the device in the first place.

## 1.6 Why Do We Need Cloud Computing?

Imagine you're starting a brand-new company. To run your business digitally, you would need: Servers, Storage, a Database, and Software.

Now you have **two choices**:

| Option | What it means |
|---|---|
| Buy everything yourself | You need to spend 20 Lakh or more upfront just to buy hardware — before you've even made a single sale |
| Rent from the cloud | You pay only for what you actually use, and you can scale up or down anytime |

This is exactly why both small startups and huge enterprises prefer cloud computing — it removes the need for a massive upfront investment.

## 1.7 Advantages of Cloud Computing

**Advantages**

1. **Cost Saving** — You don't need to buy expensive hardware. A startup can simply rent servers instead of purchasing them, which saves a huge amount of money upfront.
2. **Access Anywhere** — You can access your files from home, college, office, or your mobile phone — the only requirement is an internet connection.
3. **Large Storage** — Need more space? Just upgrade your cloud storage plan. No need to physically buy and install a new hard disk.
4. **Automatic Backup** — Cloud providers automatically back up your data in the background, which greatly reduces the risk of losing it.
5. **High Security** — Big cloud companies (like Google, Amazon, Microsoft) invest heavily in protecting your data with strong safeguards — often much stronger than what an individual or small company could set up on their own.
6. **Easy Collaboration** — Multiple people can work on the same document at the same time — think of how Google Docs lets a whole team edit one file together, live.

## 1.8 Disadvantages of Cloud Computing

**Disadvantages**

1. **Internet dependency** — You need an internet connection to access your data. No internet means no access.
2. **Data privacy concerns** — You are trusting a third-party company with your personal or business data.
3. **Ongoing costs** — Unlike a one-time hardware purchase, cloud services usually involve continuous monthly subscription fees.
4. **Service outages** — If the cloud provider's servers go down, you may temporarily lose access to your own data.

## 1.9 Popular Cloud Services We Use Every Day

| Service | Category |
|---|---|
| Gmail | Email |
| Google Drive | Storage |
| Dropbox / OneDrive | Storage |
| YouTube / Netflix | Video Streaming |
| Spotify | Music Streaming |
| Google Photos | Photo Backup |

Notice something? We already use cloud computing constantly in daily life — we just don't always realize it.

## 1.10 Recent Trends in Cloud Computing

As more and more businesses and individuals adopt cloud computing, big providers like **Google, Microsoft, IBM, and Amazon** keep innovating to make their services faster, safer, and smarter. Here are the top 10 current trends.

### 1.10.1 Trends 1–5

1. **AI & Machine Learning** — Cloud platforms are becoming self-learning and personalized, with AI/ML also helping strengthen data privacy.
2. **Data Security** — Stronger encryption, authentication, and recovery systems to minimize data breaches and data loss.
3. **Multi & Hybrid Cloud** — Businesses spread their workloads across multiple cloud providers to reduce risk and cut costs.
4. **Low-Code / No-Code** — Tools like Figma and Zoho let people build apps and websites quickly, without deep coding knowledge.

5. **Edge Computing** — Data is processed closer to where it is generated (instead of a faraway server) to reduce delay — this is helped by the rise of 5G networks.

### 1.10.2 Trends 6–10

6. **IoT (Internet of Things)** — Sensors and smart devices generate massive amounts of data, which is stored and analyzed using cloud servers.
7. **Kubernetes & Docker** — Tools that automate the scaling, deployment, and management of software containers for large applications.
8. **Serverless Computing** — A pay-as-you-go model where you don't have to manage servers yourself; the system scales automatically based on demand.
9. **DevSecOps** — Security is built directly into the software development process, to defend against cloud-based cyberattacks.
10. **Disaster Recovery** — Cloud backups help protect important data from outages, cyberattacks, and hardware failures.

*Other emerging trends to keep an eye on: Blockchain, Cloud Gaming, and Quantum Computing.*

## 1.11 The Evolution of Cloud Computing

*From Mainframes in the 1950s to the Modern Cloud — 8 Generations of Innovation*

### 1.11.1 A 70-Year Journey

Cloud computing did not appear overnight — it evolved slowly over roughly 70 years. Each new generation of technology fixed the biggest weakness of the previous one, and eventually, all these ideas combined into what we now call "the cloud."

The 8 generations, in order, are shown below.

| # | Generation | Time Period |
|---|---|---|
| 1 | Mainframe Computing | 1950 – 1970 |
| 2 | Distributed Computing | 1970 – 1980 |
| 3 | Cluster Computing | 1980 – 1990 |
| 4 | Grid Computing | 1990 – 2000 |
| 5 | Utility Computing | Late 1990s – 2000s |
| 6 | Virtualization | 1980 – Present |
| 7 | Web 2.0 | ∼2004 |
| 8 | Service Orientation | 2000s |

Let's go through each one, one at a time.

### 1.11.2 Mainframe Computing (1950 – 1970)

Mainframes were the very first "big iron" computers — extremely powerful and reliable machines, built to handle huge amounts of input and output data with almost zero downtime.

**The problem:** They pushed computing power far beyond what was previously possible, but they were **so expensive** that only large institutions (banks, governments, airlines) could afford them.

**Real-world examples:**

- **Banking Systems** — used for overnight batch-processing of millions of transactions
- **SABRE Airline System** — one of the first mainframe-powered flight reservation systems
- **Government Census** — large-scale data processing for national records

**What happened next:** This cost problem is exactly what pushed the industry toward finding a cheaper alternative — cluster computing.

### 1.11.3 Distributed Systems (1970 – 1980)

Distributed systems combined multiple independent computers that worked together but appeared to the user as a single system.

**What they introduced:**

- Scalability (could grow bigger)
- Concurrency (multiple things happening at once)
- Continuous availability (did not fully stop working if one part failed)
- Independence from failures
- Machines did not need to sit in the same physical location — they simply needed to communicate over a network

**Real-world examples:**

- **ARPANET** — an early academic network linking computers that were geographically far apart (this eventually grew into the modern internet)
- **File-Sharing Networks** — multiple machines sharing resources as though they were one logical system
- **Multi-Site Research Labs** — universities pooling computing power across different campuses

**What happened next:** This model later evolved further into cluster and grid computing.

### 1.11.4 Cluster Computing (1980 – 1990)

Cluster computing appeared as a **cheaper alternative to mainframes**.

**How it worked:** Regular machines were linked together using high-bandwidth networks. Together, they delivered computing power comparable to a mainframe, and you could easily add more machines ("nodes") whenever you needed more power.

**The problem it solved:** Cost — clusters were far cheaper than mainframes.

**The problem it still had:** The machines still needed to be **geographically close** to each other (usually in the same room or building).

**Real-world examples:**

- **Beowulf Clusters** — ordinary, off-the-shelf PCs networked together for scientific research
- **University Research Labs** — used low-cost clusters instead of buying expensive mainframes
- **Early Web Server Farms** — clustered machines sharing the load of website traffic

### 1.11.5 Grid Computing (1990 – 2000)

Grid computing solved the "geography problem" of clusters by connecting systems in **completely different geographic locations** via the internet — forming a "grid" across multiple organizations.

**The problem it solved:** Machines no longer needed to be physically close together.

**The new problem it created:** Limited bandwidth (internet speed) over such long distances.

**Core Concept**

Grid computing is often called the **direct predecessor of cloud computing** — meaning it is the technology that led most directly into what we now call "the cloud."

**Real-world examples:**

- **SETI@home** — volunteers around the world donated their computer's idle processing power to help analyze radio signals from space
- **CERN LHC Computing Grid** — physics data from the Large Hadron Collider processed across dozens of countries
- **Folding@home** — distributed protein-folding research spread across the globe

### 1.11.6 Utility Computing (Late 1990s – 2000s)

Utility computing introduced a completely new way of *paying* for computing.

**The core idea:** Compute power, storage, and infrastructure are billed on a **pay-per-use basis** — exactly like how you pay for electricity or water at home. You only pay for what you actually use.

**Why this matters:** This "pay only for what you use" idea became one of the foundational principles behind modern cloud pricing (the same model AWS, Azure, and Google Cloud use today).

**Real-world examples:**

- **Sun Grid Utility** — an early pay-per-CPU-hour computing service
- **IBM On-Demand Computing** — metered enterprise computing resources
- **Precursor to AWS Pricing** — this model set the template for today's pay-as-you-go cloud billing

### 1.11.7 Virtualization (1980 – Present)

Virtualization creates a **virtual layer** over physical hardware, which allows multiple independent "virtual machines" to run at the same time on a single physical machine.

**Why it's important:** Even though this technology was introduced decades ago, it is still the key technology underlying modern cloud platforms today. In fact, hardware virtualization remains the most common form used right now.

**Real-world examples:**

- **Amazon EC2** — thousands of virtual machines run on shared physical servers
- **VMware vCloud** — an enterprise-grade hardware virtualization platform
- **VirtualBox / Hyper-V** — widely used desktop and enterprise virtualization tools

### 1.11.8 Web 2.0 (Gained Popularity ~2004)

Web 2.0 is best understood as the **interface layer** — it is how cloud services actually interact with everyday users.

**The shift it represents:** Moving from static, unchanging web pages to interactive, dynamic web experiences.

**Why it matters:** Web 2.0 is the technology that made modern social media possible, and it greatly increased flexibility across the entire web.

**Real-world examples:**

- **Google Maps** — interactive, dynamic mapping right inside your browser
- **Facebook** — social media built entirely on interactive web pages
- **Twitter** — real-time, dynamic content sharing at a global scale

### 1.11.9 Service Orientation (2000s)

Service orientation became the **reference model** for cloud computing — meaning it is the blueprint that modern cloud services are still built on.

**What it introduced:**

- **Quality of Service (QoS)** with **Service Level Agreements (SLAs)** — formal promises about uptime and performance
- **Software as a Service (SaaS)** — software delivered as an online service instead of something you install

**Real-world examples:**

- **Salesforce** — one of the first major SaaS companies, offering contractual service guarantees
- **SLA-Backed Hosting** — providers that formally guarantee uptime and performance
- **Early Web Services (SOAP/XML)** — standardized, service-oriented application architecture

### 1.11.10 All Roads Lead to the Cloud

Putting it all together:

**Core Concept**

**Cloud Computing** means storing and accessing data and programs on **remote servers hosted on the internet**, instead of on a computer's hard drive or a local server.

Each generation contributed one essential ingredient to modern cloud computing:

| Generation | What it contributed |
|---|---|
| Mainframe | Raw computing power |
| Distributed | Remote access |
| Virtualization | Virtualized resources |
| Utility Computing | Pay-per-use billing |
| Service Orientation | SaaS & SLAs |

**Bottom line:** Today's cloud platforms (AWS, Azure, Google Cloud) are essentially the **sum of every generation that came before them.**

## 1.12 Public vs Private vs Hybrid Cloud

*Choosing the Right Cloud Deployment Model*

### 1.12.1 Three Deployment Models, One Decision

The main question that separates these three models is:

> Who owns the infrastructure — and who else shares it with you?

| Model | Meaning | Tenancy |
|---|---|---|
| Public Cloud | Shared infrastructure owned by a third-party provider, pay-as-you-go | Multi-Tenant |

| Model | Meaning | Tenancy |
|---|---|---|
| Private Cloud | Dedicated infrastructure for one single organization, maximum control | Single-Tenant |
| Hybrid Cloud | A strategic mix of both — sensitive workloads stay private, the rest goes public | Mixed |

### 1.12.2 Public Cloud — Owned & Shared by a Provider

**Definition:** Infrastructure and services that are fully owned, operated, and maintained by a third-party provider, delivered over the public internet, and shared among multiple organizations at once. This sharing setup is called **multi-tenancy**.

**Core Concept**

You pay only for the exact resources you consume.

**Well-known examples:** Amazon Web Services (AWS), Microsoft Azure, Google Cloud Platform (GCP)

**Advantages**

- Cost-efficient — no hardware to buy, purely pay-as-you-go
- Automatic updates, patches & maintenance handled by the provider
- High accessibility — usable from anywhere with an internet connection

**Disadvantages**

- Security & privacy concerns since servers are shared with others
- Limited control over the underlying infrastructure
- Performance depends on reliable internet connectivity
- May not meet strict compliance requirements (for example, HIPAA in healthcare)
- Unmonitored usage can cause unexpected cost overruns

### 1.12.3 Private Cloud — Dedicated to One Organization

**Definition:** A computing environment dedicated entirely to a single organization. It can be hosted on-site (inside the company itself) or by a third party — but the key point is that resources are **never shared** with other tenants.

**Core Concept**

Maximum control and isolation for sensitive operations.

**Well-known examples:** VMware, HPE / Dell Infrastructure, Bespoke Internal Data Centers

**Advantages**

- Higher security — full control to customize protections
- Deep customization of infrastructure and services
- Greater privacy over exactly who accesses data and resources

**Disadvantages**

- Higher cost — dedicated hardware & networking is expensive
- Limited scalability — capped by the physical hardware you own
- Technical complexity — requires specialized IT expertise to manage
- Still vulnerable to breaches and cyberattacks (private does not automatically mean safe)
- Lack of standardization can complicate future migration
- Ongoing maintenance and upgrades require real, continuous resources

### 1.12.4 Hybrid Cloud — The Best of Both Worlds

**Definition:** Combines public and private cloud environments, allowing data and applications to move between them. Sensitive, mission-critical workloads run on the private cloud, while the public cloud absorbs burst traffic or less-sensitive workloads.

**Core Concept**

Optimizing for security, performance, and cost — all at the same time.

**Well-known examples:** AWS Outposts, Azure Stack, IBM Hybrid Cloud

**Advantages**

- Flexibility — sensitive data stays private, while the public cloud adds extra reach
- Scalability — workloads can shift between the two clouds on demand
- Balanced security combined with the public cloud's cost savings

**Disadvantages**

- Complexity of integrating two very different environments
- Often costlier overall than using just one model alone
- Security risks can appear at the "seams" between environments
- Data governance challenges across environments (e.g., GDPR, HIPAA compliance)
- Network latency (delay) between environments
- Integration & compatibility challenges
- Possible vendor lock-in when using multiple providers

### 1.12.5 Head-to-Head Comparison

| Factor | Public Cloud | Private Cloud | Hybrid Cloud |
|---|---|---|---|
| Resource Allocation | Shared / multi-tenant | Dedicated / single-tenant | Mix of shared & dedicated |
| Data Storage | Provider's remote servers | Internal, isolated servers | Sensitive data private, rest public |
| Pricing Model | Pay-as-you-go, variable | High fixed upfront cost | Mixed: fixed + variable |
| Management | Fully by provider | Internal IT team | Co-managed |
| Scalability | Near-infinite, instant | Limited by hardware | Flexible — bursts to public |
| Overall Cost | Cost-effective at scale | Expensive to build/maintain | Optimized by workload routing |

### 1.12.6 Which One Should You Choose?

There is **no single "best" model** — the right choice always depends on your specific workload sensitivity, budget, and need for scale.

- **Choose Public Cloud if:** You're a startup, have variable workloads, and want zero hardware overhead.
- **Choose Private Cloud if:** You're in a regulated industry, handle sensitive data, or need full control over your infrastructure.
- **Choose Hybrid Cloud if:** Your business needs to balance sensitive workloads with the ability to scale on demand.

### 1.12.7 Quick Revision Summary

- **Cloud Computing** = using storage/software/servers/databases over the internet instead of your own computer.
- Your data (photos, emails) is stored on remote **servers** in **data centers**, not on your device.
- **Advantages:** cost saving, access anywhere, large storage, auto backup, high security, easy collaboration.
- **Disadvantages:** needs internet, privacy concerns, ongoing costs, possible outages.
- **8 generations that led to cloud computing:** Mainframe → Distributed → Cluster → Grid → Utility → Virtualization → Web 2.0 → Service Orientation.
- **Grid computing** = direct predecessor of cloud computing.
- **Three deployment models:**
  - **Public** = shared, cheap, less control (AWS, Azure, GCP)
  - **Private** = dedicated, expensive, full control (VMware, internal data centers)
  - **Hybrid** = mix of both (AWS Outposts, Azure Stack, IBM Hybrid Cloud)

## 1.13 Categories of Cloud Computing: Everything as a Service

We have already seen that cloud computing allows us to use computing resources through the Internet. But what exactly can we get from the cloud?

Sometimes we need only a server or storage. Sometimes we need a complete platform for developing an application. In other cases, we simply want to use ready-made software.

This leads to the idea of **"Everything as a Service"** or **XaaS (Anything as a Service)**.

The main service models discussed here are:

1. **Infrastructure as a Service (IaaS)**
2. **Platform as a Service (PaaS)**
3. **Software as a Service (SaaS)**
4. **Security as a Service (SECaaS)**

The easiest way to understand these models is to think about **how much work is done by the user and how much is managed by the cloud provider**.

### 1.13.1 Infrastructure as a Service (IaaS)

**What is IaaS?**

Imagine that a company wants to start a website. It needs a server, storage, networking and other computing resources. Buying all this physical hardware can be expensive.

Instead, the company can rent these resources from a cloud provider. This is called **Infrastructure as a Service (IaaS)**.

**Core Concept**

**IaaS provides basic computing infrastructure such as servers, storage and networking through the Internet.**

The cloud provider takes care of the physical hardware. The user can manage things such as the operating system, applications and data.

#### 1.13.1.1 What Does IaaS Provide?

- Virtual machines
- Servers
- Storage
- Networking
- Virtual networks
- Firewalls

- Load balancers

#### 1.13.1.2 Examples of IaaS

- Amazon EC2
- Microsoft Azure Virtual Machines
- Google Compute Engine

#### 1.13.1.3 Advantages of IaaS

**Advantages**

- No need to buy expensive physical servers.
- Resources can be increased or decreased easily.
- Users generally pay only for the resources they use.
- It is suitable for large and changing workloads.
- It gives users considerable control over their computing environment.

#### 1.13.1.4 Simple Example

Suppose a company needs a server for its website.

Instead of buying a physical server, the company can **rent a virtual server from a cloud provider**. If the website becomes popular and needs more resources, the company can increase them.

**Remember: IaaS = Rent the infrastructure.**

### 1.13.2 Platform as a Service (PaaS)

**What is PaaS?**

Now imagine that you are a programmer. You want to develop an application, but you do not want to spend time installing servers, operating systems, databases and development environments.

A cloud provider can give you a **ready-made development environment**. This is called **Platform as a Service (PaaS)**.

**Core Concept**

**PaaS provides a ready-to-use platform for developing, testing and deploying applications.**

The cloud provider manages most of the underlying infrastructure. Developers can mainly concentrate on writing and managing their application code.

#### 1.13.2.1 What Does PaaS Provide?

- Development tools

- Runtime environments
- Databases
- Middleware
- Operating systems
- Development frameworks
- Application hosting environments

#### 1.13.2.2 Examples of PaaS

- Google App Engine
- AWS Elastic Beanstalk
- Microsoft Azure App Service
- Heroku

#### 1.13.2.3 Advantages of PaaS

**Advantages**

- Makes application development easier.
- Saves development time.
- Developers do not need to manage physical servers.
- Application deployment becomes easier.
- Supports development teams.
- Resources can be scaled when required.

#### 1.13.2.4 Simple Example

Suppose a group of students wants to develop an **online shopping application**.

They do not want to spend time setting up servers and operating systems. They can use a PaaS platform and concentrate mainly on **developing the application**.

**Remember: PaaS = Rent the development platform.**

### 1.13.3 Software as a Service (SaaS)

**What is SaaS?**

Now think about applications such as **Gmail or Google Docs**.

Do we need to buy a server?

Do we need to install the operating system?

Do we need to maintain the application ourselves?

**No.**

We simply open the application and use it through the Internet.

This is called **Software as a Service (SaaS)**.

**Core Concept**

**SaaS provides complete software applications to users through the Internet.**

The cloud provider manages the application, servers, storage, updates and maintenance.

#### 1.13.3.1 Examples of SaaS

- Gmail
- Google Docs
- Microsoft 365
- Salesforce
- Zoom
- Slack

#### 1.13.3.2 Advantages of SaaS

**Advantages**

- No need to install the software locally.
- Can be accessed through the Internet.
- Software updates are handled by the provider.
- Very little maintenance is required from the user.
- Can be accessed from different devices.
- Usually available through subscription or usage-based plans.

#### 1.13.3.3 Simple Example

Suppose you want to prepare a document.

You open **Google Docs** in your browser and start typing. You do not have to worry about where the servers are located or how the software is updated.

The cloud provider takes care of these things.

**Remember: SaaS = Use ready-made software.**

### 1.13.4 Security as a Service (SECaaS)

**Why do we need Security as a Service?**

Cloud computing is useful, but **security is very important**. Organizations need to protect their data, applications, networks, users and devices.

Setting up and maintaining all security systems by themselves can be expensive and difficult. Instead, organizations can obtain security services from cloud providers.

This is called **Security as a Service (SECaaS)**.

**Core Concept**

**Security as a Service provides security functions through the cloud to protect systems, applications, networks and data.**

#### 1.13.4.1 What Can SECaaS Provide?

- Antivirus and malware protection
- Firewall services
- User authentication
- Identity and access management
- Intrusion detection
- Intrusion prevention
- Security monitoring
- Data encryption
- Vulnerability scanning
- Security event monitoring

#### 1.13.4.2 Advantages of SECaaS

**Advantages**

- Reduces the cost of maintaining security systems.
- Provides access to specialized security tools.
- Security services can be scaled when required.
- Provides centralized security monitoring.
- Security updates can be managed by the provider.
- Useful for organizations that do not have a large security team.

#### 1.13.4.3 Simple Example

Imagine a college has hundreds of computers connected to its network.

Instead of installing and maintaining separate security systems for everything, the college can use a **cloud-based security service** to monitor the network and identify suspicious activities.

**Remember: SECaaS = Get security through the cloud.**

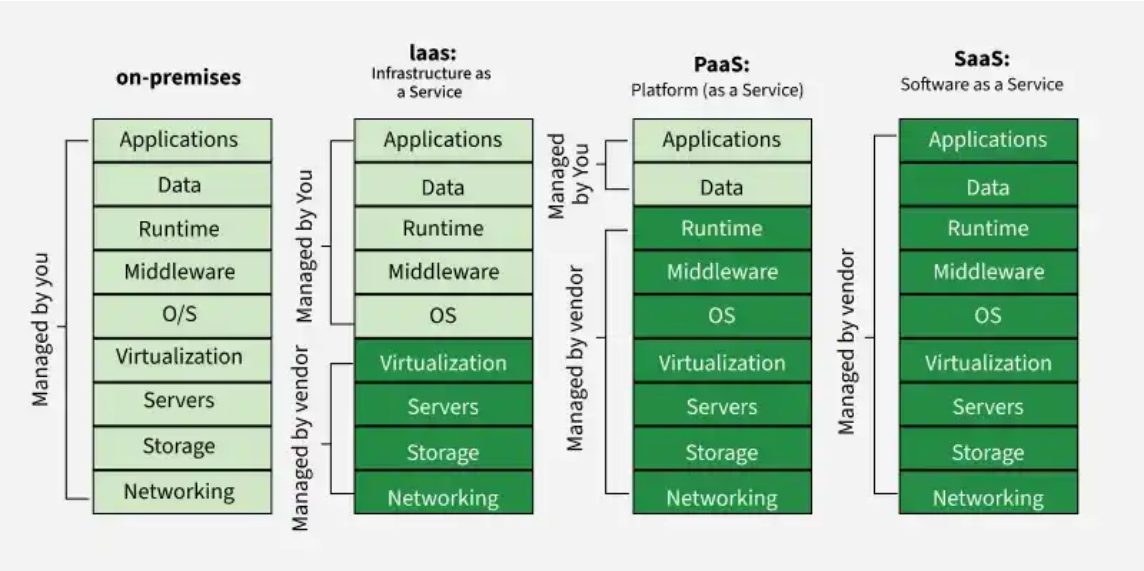



Figure 1.1: Management responsibilities in On-Premises, IaaS, PaaS and SaaS

**Understanding the Figure:**
The figure shows how the responsibility is divided between the user and the cloud service provider.
In an **on-premises** system, the organization manages almost everything, including networking, storage, servers, virtualization, operating systems and applications.
In **IaaS**, the cloud provider manages the basic infrastructure, while the user manages the operating system, middleware, runtime, data and applications.
In **PaaS**, the cloud provider manages more of the system, including the operating system, middleware and runtime. The user mainly manages the application and data.
In **SaaS**, the cloud provider manages almost the entire stack. The user mainly uses the application.

## 1.14 Understanding the Difference Between IaaS, PaaS and SaaS

The easiest way to understand the difference is to ask:

**How much do I manage, and how much does the cloud provider manage?**

| Feature | IaaS | PaaS | SaaS |
|---|---|---|---|
| Main purpose | Provides infrastructure | Provides a development platform | Provides complete software |
| User control | More control | Moderate control | Very little technical control |

| Feature | IaaS | PaaS | SaaS |
|---|---|---|---|
| User mainly manages | Operating system, applications and data | Application code and data | Mainly the use of the application |
| Main users | IT administrators and developers | Software developers | End users |
| Example | Amazon EC2 | Google App Engine | Google Docs |

## 1.15 An Easy Way to Remember IaaS, PaaS and SaaS

Think about preparing food.

- **IaaS:** You get the kitchen and equipment. You prepare most things yourself.
- **PaaS:** You get a ready kitchen with many things already arranged. You mainly concentrate on preparing the food.
- **SaaS:** You simply order the finished food and eat it.

The same idea can be applied to cloud computing:

**Core Concept**

- **IaaS** → Infrastructure
- **PaaS** → Platform
- **SaaS** → Software
- **SECaaS** → Security

## 1.16 Quick Revision: Cloud Service Models

| Service | What does it provide? | Easy way to remember |
|---|---|---|
| **IaaS** | Servers, storage and networking | **Rent infrastructure** |
| **PaaS** | Development platform and tools | **Develop applications** |
| **SaaS** | Complete software applications | **Use the software** |
| **SECaaS** | Security services and protection | **Protect systems and data** |

**Core Concept**

**One-line summary:**
IaaS provides the **infrastructure**, PaaS provides the **platform**, SaaS provides the **software**, and SECaaS provides **security** through the cloud.

## 1.17 Pros and Cons of Cloud Computing

Cloud computing provides computing resources such as servers, storage, databases, networking, and software through the Internet instead of requiring users to maintain all computing resources locally. It provides flexibility and scalability, but it also introduces challenges related to security, cost, connectivity, and dependence on service providers.

### 1.17.1 Advantages of Cloud Computing

#### 1.17.1.1 Cost Reduction

Cloud computing reduces the need to purchase and maintain expensive hardware and software.

- No need for large initial investment in servers.
- Maintenance costs are reduced.
- Users generally pay only for the resources they use.
- Suitable for startups, educational institutions, and small businesses.

**Example:** A company can rent cloud storage instead of purchasing and maintaining its own storage servers.

#### 1.17.1.2 Scalability

Cloud resources can be increased or decreased according to workload requirements.

- Resources can be added during periods of high demand.
- Resources can be reduced when demand decreases.
- Helps organizations handle changing workloads efficiently.

**Example:** An online shopping website can increase computing resources during a festival sale.

#### 1.17.1.3 Accessibility

Cloud services can be accessed from different locations using an Internet connection.

- Supports remote work.
- Users can access files and applications from multiple devices.
- Facilitates collaboration among geographically distributed teams.

**Example:** Students and teachers can access educational materials from home or college.

#### 1.17.1.4 Flexibility

Cloud computing allows organizations to quickly select and configure the resources they need.

- Computing power can be changed easily.
- Storage capacity can be increased when required.
- Different services can be selected according to business needs.

#### 1.17.1.5 High Availability

Many cloud providers design their infrastructure with redundant servers and systems.

- Services can continue even when some hardware fails.
- Data may be replicated across multiple systems or locations.
- Improves service availability.

#### 1.17.1.6 Backup and Disaster Recovery

Cloud platforms can be used to store backup copies of data.

- Protects data against hardware failures.
- Helps recover data after accidental deletion or disasters.
- Reduces the need for maintaining separate physical backup infrastructure.

#### 1.17.1.7 Automatic Updates

Cloud service providers generally manage many software and infrastructure updates.

- Reduces administrative workload.
- Users can receive updated versions of services.
- Security patches can be deployed by the provider.

#### 1.17.1.8 Faster Deployment

Cloud resources can usually be provisioned much faster than traditional physical infrastructure.

**Example:** A developer can create a virtual server in minutes instead of purchasing and configuring a physical server.

### 1.17.2 Disadvantages of Cloud Computing

#### 1.17.2.1 Internet Dependency

Cloud services normally require a reliable Internet connection.

- Poor connectivity can affect performance.
- Internet failure may prevent access to cloud resources.

- Network latency can affect applications requiring real-time responses.

#### 1.17.2.2 Security and Privacy Risks

Data stored in the cloud is managed using infrastructure controlled by a cloud provider. Potential security and privacy concerns include:

- Unauthorized access
- Data breaches
- Account compromise
- Incorrect access permissions
- Data privacy issues

Therefore, proper authentication, encryption, and access control are important.

#### 1.17.2.3 Vendor Lock-In

Organizations may become dependent on a particular cloud provider.

- Moving applications to another provider may be difficult.
- Data migration can be expensive or time-consuming.
- Different providers may use different technologies and interfaces.

**Example:** An application designed heavily around one provider's services may require significant modification before it can run elsewhere.

#### 1.17.2.4 Limited Control

In cloud computing, the provider manages much of the underlying infrastructure. Users may have limited control over:

- Physical servers
- Data-center infrastructure
- Hardware configuration
- Certain security mechanisms

#### 1.17.2.5 Downtime

Cloud providers can experience service interruptions.

Even though cloud platforms generally provide high availability, failures can occur because of:

- Hardware problems
- Software failures
- Network problems
- Configuration errors

- Cyberattacks

#### 1.17.2.6 Unpredictable Costs

Although cloud computing can reduce initial investment, poorly managed usage can result in unexpectedly high bills.

For example:

- Running unnecessary virtual machines
- Excessive storage usage
- High data-transfer usage
- Unused cloud resources

Therefore, cloud cost monitoring is important.

#### 1.17.2.7 Performance Issues

Cloud performance may vary depending on:

- Network bandwidth
- Server load
- Distance from the data center
- Type of cloud service
- Resource allocation

Applications requiring very low latency may not always perform optimally in a public cloud.

#### 1.17.2.8 Compliance and Legal Issues

Organizations handling sensitive or regulated data must comply with applicable laws and regulations.

Issues may include:

- Where data is physically stored
- Who can access the data
- Data retention requirements
- Industry-specific regulations

### 1.17.3 Advantages and Disadvantages at a Glance

### 1.17.4 Simple Example

Consider a college that needs an online learning system.

Table 1.9: Advantages and Disadvantages of Cloud Computing

| **Advantages** | **Disadvantages** |
|---|---|
| Lower initial cost | Internet dependency |
| Easy scalability | Security and privacy concerns |
| Accessible from anywhere | Vendor lock-in |
| Flexible resource management | Limited infrastructure control |
| High availability | Possibility of downtime |
| Backup and disaster recovery | Unpredictable costs |
| Automatic updates | Possible performance issues |
| Faster deployment | Compliance challenges |

#### 1.17.4.1 Traditional Approach

The college must purchase and maintain:

- Servers
- Storage devices
- Networking equipment
- Backup systems

It must also manage hardware maintenance and upgrades.

#### 1.17.4.2 Cloud Approach

The college can use cloud-based servers and storage.

**Benefits:**

- Lower initial investment
- Easy expansion when the number of students increases
- Remote access
- Backup facilities

**Challenges:**

- Requires Internet connectivity
- Student data must be properly protected
- Monthly usage costs must be monitored
- The college may become dependent on the selected provider

## 1.18 What Is Virtualization?

Virtualization is a computer architecture technology that allows multiple virtual machines (VMs) to be multiplexed on the same physical hardware. The concept dates back to the 1960s. Its purpose is to enhance resource sharing among many users and to improve computer performance in terms of resource utilization and application flexibility.

Hardware resources (CPU, memory, I/O devices, etc.) or software resources (operating system and software libraries) can be virtualized at various functional layers. The core idea is to separate the hardware from the software to yield better system efficiency — the same principle behind virtual memory, which gave users access to a much larger memory space than physically existed.

Before virtualization: a traditional computer runs a single host operating system tailored to its hardware.
After virtualization: a software layer called the hypervisor / Virtual Machine Monitor (VMM) is inserted between hardware and the OS, allowing multiple guest operating systems (and their applications) to run independently on the same physical machine.

**Figure 3.1 — Architecture Before and After Virtualization**

Figure 3.1Architecture of a computer system before and after virtualization (VMM = Virtual Machine Monitor).

According to a 2009 Gartner Report, virtualization was rated the top strategic technology poised to change the computer industry. Given sufficient storage, any computer platform can, in principle, be installed on another host computer — even one using a different instruction set and running a different operating system.

## 1.19 Levels of Virtualization Implementation

The virtualization layer creates the abstraction of VMs by being interposed at different levels of a computer system. There are five common levels at which virtualization can be implemented:

- Instruction Set Architecture (ISA) level
- Hardware Abstraction Level (HAL)
- Operating System level
- Library Support level
- User-Application level

**Figure 3.2 — The Five Levels of Virtualization**

Figure 3.2Virtualization ranging from the hardware to the application level (the diagram you already have — Application / Library / OS / HAL / ISA levels with examples such as JVM, WINE, Jail, VMware, Bochs).

### 1.19.1 Instruction Set Architecture (ISA) Level

At the ISA level, virtualization works by emulating a given instruction set using the ISA of the host machine. For example, MIPS binary code can be run on an x86-based host with the help of ISA emulation. This makes it possible to run large amounts of legacy binary code — written for other processors — on any new hardware host.

- Basic emulation is done through code interpretation: an interpreter converts source instructions to target instructions one at a time (one source instruction may need tens or hundreds of native target instructions), which is relatively slow.
- For better performance, **dynamic binary translation** is used — this translates whole basic blocks (or larger program traces/super blocks) of source instructions to target instructions at once, improving efficiency.
- This approach requires adding a processor-specific software translation layer to the compiler, producing a Virtual Instruction Set Architecture (V-ISA).

### 1.19.2 Hardware Abstraction Level (HAL)

Hardware-level virtualization is performed directly on top of the bare hardware. It generates a virtual hardware environment for a VM and simultaneously manages the underlying real hardware. The goal is to virtualize resources — processors, memory, and I/O devices — so that hardware utilization is improved when many users work concurrently.

- First implemented in the IBM VM/370 in the 1960s.
- More recently, the Xen hypervisor has been used to virtualize x86-based machines to run Linux or other guest operating systems.

### 1.19.3 Operating System Level

This is an abstraction layer inserted between the traditional OS and user applications. OS-level virtualization creates isolated containers on a single physical server, along with OS instances, to make full use of the hardware and software in data centers.

- The containers behave like real, independent servers.
- Commonly used to create virtual hosting environments that allocate hardware resources among many mutually distrusting users (e.g., web hosting).
- Also used, to a lesser extent, for server consolidation — moving services from separate physical hosts into containers or VMs on one server.

### 1.19.4 Library Support Level

Most applications use APIs exported by user-level libraries rather than making lengthy system calls directly to the OS. Since these APIs are well documented, they become a natural candidate for virtualization.

- Virtualization is achieved by controlling the communication link between applications and the rest of the system through API hooks.
- **Example — WINE**: implements this approach to let Windows applications run on top of UNIX hosts.
- **Example — vCUDA**: allows applications running inside VMs to leverage GPU hardware acceleration.

### 1.19.5 User-Application Level

Application-level virtualization virtualizes an application as a VM. Since an application normally runs as a process on a traditional OS, this is also known as process-level virtualization.

- The most popular approach is deploying High-Level Language (HLL) VMs, where the virtualization layer sits as a program on top of the OS and exports an abstract machine that runs programs compiled for it.
- Examples: Microsoft .NET CLR and the Java Virtual Machine (JVM).
- Other forms are called application isolation, application sandboxing, or application streaming — the application is wrapped in a layer isolated from the host OS and other applications, making it easy to distribute and remove.
- Example: the LANDesk application virtualization platform, which deploys software as self-contained executables that need no installation, system modification, or elevated privileges.

### 1.19.6 Relative Merits of Different Virtualization Levels

The table below compares the levels on four technical merits: Higher Performance, Application Flexibility, Implementation Complexity, and Application Isolation. More X marks (up to five) means a stronger advantage on that merit.

tableheadfill

| | | | | |
|---|---|---|---|---|
| ISA Level | X | XXXXX | XXXXX | XX |
| Hardware Abstraction Level | XXXXX | XX | XXXXX | XXX |
| Operating System Level | XXXXX | XX | XX | XX |
| Library (User-Level API) Level | XXX | XX | XX | XX |
| User Application Level | XX | XXX | XXX | XXXXX |

Overall, hardware- and OS-level support tend to yield the highest performance, but they are also the most expensive to implement. User isolation is hardest to achieve at the OS level. ISA-level implementation offers the best application flexibility.

## 1.20 VMM Design Requirements and Providers

Hardware-level virtualization inserts a layer — the Virtual Machine Monitor (VMM), also called a hypervisor — between the real hardware and traditional operating systems.

The VMM manages the hardware resources of the computing system and captures every attempt by a program to access the hardware, effectively acting like a traditional OS. A single hardware component (e.g., the CPU) can be virtualized into several virtual copies, allowing several traditional operating systems — the same or different — to run simultaneously on the same hardware.

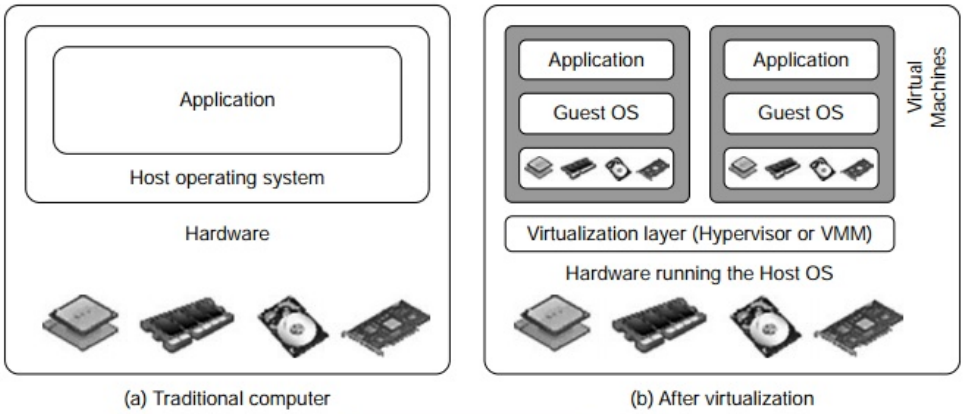



**FIGURE 3.1**

The architecture of a computer system before and after virtualization, where VMM stands for virtual machine monitor.

Figure 1.2: VMM/hypervisor layered between physical hardware and multiple guest OS + application stacks.

### Three Requirements of a VMM

**1. Equivalence / Fidelity** — A VMM should provide an environment for programs that is essentially identical to the original physical machine.

**2. Performance** — Programs running in this environment should show, at worst, only minor decreases in speed.

**3. Resource Control** — A VMM must be in complete control of the system resources.

Two exceptions to the "identical environment" rule are permitted: (a) differences caused by the availability of system resources when more than one VM runs on the same machine (the sum of resources given to all VMs is greater than the real machine's total), and (b) differences caused by timing dependencies, due to the extra software layer and other VMs running concurrently. These exceptions affect only performance, not function — which is why an ordinary time-sharing OS does not qualify as a VMM.

A VMM must also be efficient. Traditional emulators and complete software interpreters (simulators) emulate every instruction through functions or macros — the most flexible approach, but too slow for practical use as a real machine. To guarantee efficiency, a statistically dominant subset of the virtual processor's instructions must be executed directly by the real processor, without software intervention by the VMM.

### Complete Control of Resources

- The VMM is responsible for allocating hardware resources to programs.

- A program cannot access any resource that has not been explicitly allocated to it.
- Under certain circumstances, the VMM can regain control of resources already allocated.

Not every processor architecture satisfies these requirements naturally. For example, the x86 architecture has historically had difficulty trapping on some privileged instructions. When a processor is not designed to support virtualization directly, the hardware itself must be modified — this is known as **hardware-assisted virtualization**.

## 1.21 Virtualization Support at the OS Level

VM technology underpins cloud computing, which shifts the hardware and staffing costs of running a computing center to a third party — much like a bank. Cloud computing faces two key challenges that OS-level virtualization helps address:

- The need to use a variable number of physical machines and VM instances depending on workload — a task may need only one CPU at times and hundreds at others.
- The slow operation of instantiating new VMs — new VMs typically start either as fresh boots or as replicas of a template VM, unaware of the current application state.

### 1.21.1 Why OS-Level Virtualization?

Hardware-level VMs are slow to initialize because each VM builds its own image from scratch — a serious problem when a cloud environment may need to initialize thousands of VMs at once. Storing many VM images is also costly, since there is considerable repeated content across images. Full hardware-level virtualization additionally suffers from slower performance, lower density, and (for para-virtualization) the need to modify the guest OS. OS-level virtualization offers a lighter-weight alternative to these problems.

Operating system virtualization inserts a virtualization layer inside an OS to partition a machine's physical resources, enabling multiple isolated VMs within a single OS kernel. Such a VM is often called a virtual execution environment (VE), Virtual Private System (VPS), or simply a container.

- From the user's point of view, each VE looks like a real server, with its own processes, file system, user accounts, network interfaces/IP addresses, routing tables, and firewall rules.
- All VEs share the same OS kernel, so OS-level virtualization is also called single-OS-image virtualization.

### 1.21.2 Advantages of OS Extensions

- Minimal startup/shutdown cost, low resource requirements, and high scalability, because all OS-level VMs on a machine share a single OS kernel.

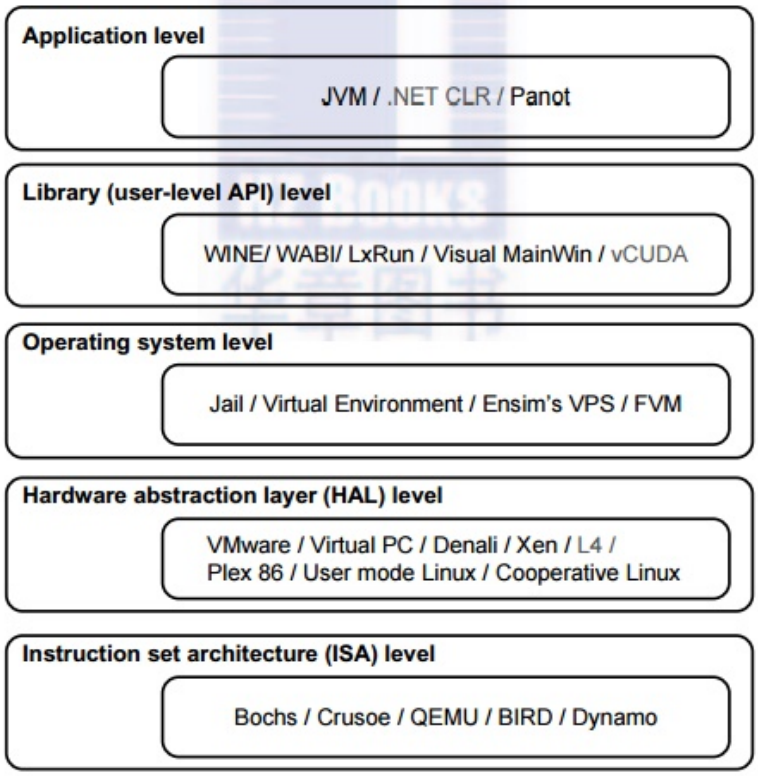



**FIGURE 3.2**

Virtualization ranging from hardware to applications in five abstraction levels.

Figure 1.3: OS-level virtualization viewed from the machine stack — a virtualization layer inside the OS partitions hardware resources among several containers/VPS.

- A VM and its host environment can synchronize state changes when necessary, since the virtualization layer lets VM processes access host resources (without modifying them).

These two benefits directly overcome the hardware-level VM problems of slow initialization and being unaware of the current application state.

### 1.21.3 Disadvantages of OS Extensions

All OS-level VMs on one physical machine must run the same operating system family as the host — for example, a Windows XP guest cannot run inside a Linux-based container. This is limiting in cloud environments where users want a free choice between Windows, Linux, and other operating systems.

To implement OS-level virtualization, isolated execution environments are created from a single OS kernel, and access requests from a VM are redirected to that VM's own resource partition. For example, the `chroot` command in UNIX creates several virtual root directories, one per VM.

- **Approach 1** — duplicate common resources into every VM partition: simple but incurs significant resource cost and overhead.

- **Approach 2** — share most resources with the host and create private copies only on demand: more efficient, and generally preferred.

### 1.21.4 Virtualization on Linux and Windows Platforms

Most reported OS-level virtualization systems are Linux-based (support on Windows remains largely at the research stage). Because new hardware may require a new/patched kernel, different Linux distributions ship patched kernels for extended functionality — though most Linux platforms are not tied to any single special kernel, so a host can run several VMs simultaneously on the same hardware.

tableheadfill

| | | |
|---|---|---|
| Linux VServer | Linux | OS-level virtualization for running multiple Linux-based guest environments |
| OpenVZ | Linux | Open-source, container-based virtualization; modifies the Linux kernel for isolation, resource management, and checkpointing |
| FVM (Feather-weight Virtual Machine) | Windows NT | OS-level virtualization built specifically for the Windows NT platform |

#### Example — OpenVZ

OpenVZ is an open-source, Linux-based, container-based virtualization tool. It modifies the Linux kernel to support virtualization, subsystem isolation, limited resource management, and checkpointing. Several Virtual Private Servers (VPSes) can run simultaneously on one physical machine, each looking like a normal Linux server with its own files, users/groups, process tree, virtual network, virtual devices, and IPC (semaphores and messages).

OpenVZ's resource-management subsystem has three components:

- **Two-level disk allocation** — the OpenVZ administrator sets total disk space per VM (level 1); the VM's own administrator then allocates that space among its users/groups (level 2).
- **Two-level CPU scheduler** — a first-level scheduler decides which VM receives the next time slice (based on virtual CPU priority/limits); a second-level scheduler within each VM behaves like a normal Linux scheduler.
- **Resource controller** — about 20 tunable parameters covering all aspects of VM operation, giving fine control over the resources a VM can use.

OpenVZ also supports checkpointing and live migration: a VM's complete state can be saved to a disk file in seconds, transferred to another physical machine, and restored there — although established network connections still introduce a brief delay during migration.

## 1.22 Middleware Support for Virtualization

Library-level virtualization — also called user-level Application Binary Interface (ABI) or API emulation — creates an execution environment for running an "alien" program on a platform, without creating a full VM to run an entire guest operating system. The key mechanism is API call interception and remapping.

| tableheadfill | |
|---|---|
| WABI (Windows ABI) | Middleware that converts Windows system calls into Solaris system calls |
| Lxrun | System-call emulator that lets Linux applications built for x86 run on UNIX systems |
| WINE | Provides library support to virtualize x86 processors, allowing Windows applications to run on UNIX hosts |
| Visual MainWin | Compiler support system for developing Windows applications (via Visual Studio) that run on certain UNIX hosts |
| vCUDA | Virtualizes the CUDA library so GPU-accelerated CUDA applications inside a guest OS can use the host's GPU |

### Example — vCUDA for GPU Virtualization

CUDA is a programming model and library for general-purpose GPU (GPGPU) computing, letting compute-intensive applications exploit GPU performance on the host OS. Running CUDA applications directly inside hardware-level VMs is difficult, so vCUDA virtualizes the CUDA library and installs it inside the guest OS. When a guest application calls the CUDA API, vCUDA intercepts the call and redirects it to the real CUDA API on the host OS.

vCUDA uses a client–server model with three user-space components:

- **vCUDA library** — resides in the guest OS as a drop-in replacement for the standard CUDA library; intercepts and redirects API calls to the stub.
- **Virtual GPU (vGPU) in the guest OS** — acts as the client; gives applications a uniform view of the underlying GPU hardware, returns a local virtual address when memory is allocated (while notifying the stub to allocate the real device memory), and records the CUDA API call flow.
- **vCUDA stub in the host OS** — acts as the server; receives and interprets remote requests, creates the execution context for each API call, returns results to the guest OS, and manages the actual physical GPU resource allocation.

## 1.23 Virtualization Structures, Tools, and Mechanisms

Before virtualization, the operating system manages the hardware directly. After virtualization, a virtualization layer is inserted between the hardware and the operating system. This layer is responsible for converting portions of the real hardware into virtual hardware, so that different operating systems — such as Linux and Windows — can run on the same physical machine simultaneously.

Depending on where the virtualization layer sits, there are three typical classes of VM architecture:

- **Hypervisor architecture** — the virtualization layer (hypervisor / VMM) sits directly on the bare hardware.
- **Para-virtualization** — the guest OS is modified to cooperate with the virtualization layer.
- **Host-based virtualization** — the virtualization layer sits on top of an existing host operating system.

The hypervisor is also known as the VMM (Virtual Machine Monitor); both terms describe the same virtualization function.

### 1.23.1 Hypervisor and Xen Architecture

The hypervisor supports hardware-level virtualization on bare-metal devices like the CPU, memory, disk, and network interfaces. The hypervisor software sits directly between the physical hardware and the guest OS. This virtualization layer is referred to as either the VMM or the hypervisor, and it provides *hypercalls* for the guest OSes and applications.

Depending on functionality, a hypervisor can assume one of two architectures:

- **Micro-kernel hypervisor** (e.g., Microsoft Hyper-V) — includes only the basic and unchanging functions, such as physical memory management and processor scheduling. Device drivers and other changeable components sit outside the hypervisor, so its code size is smaller.
- **Monolithic hypervisor** (e.g., VMware ESX for server virtualization) — implements all the aforementioned functions, *including* the device drivers, making its code base larger.

A hypervisor must be able to convert physical devices into virtual resources dedicated to the VM that is using them.

#### Architecture — Hypervisor Sitting Between Hardware and Guest OSes

#### The Xen Architecture

The core components of a Xen system are the hypervisor, kernel, and applications. Many guest OSes can run on top of the Xen hypervisor, but not all guest OSes are created equal — one in particular controls the others.

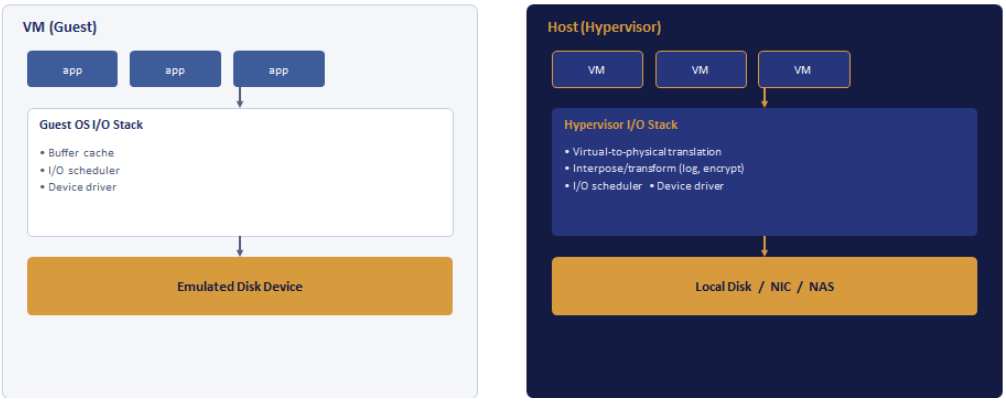



Figure 1.4: Generic hypervisor/VMM architecture — several guest OS + application stacks running on top of the hypervisor, which sits directly on the physical hardware.

- **Domain 0** — the privileged guest OS of Xen. It is the first domain loaded when Xen boots, before any file-system drivers are available. Domain 0 is designed to access hardware directly and manage devices.
- **Domain U** — the unprivileged guest domains. Domain 0 is responsible for allocating and mapping hardware resources for the guest domains (Domain U).

### 1.23.2 Para-Virtualization with Compiler Support

Para-virtualization requires modifying the guest operating system. A para-virtualized VM provides special APIs that require substantial OS modifications in user applications. Performance degradation is a critical issue of a virtualized system — no one wants to use a VM if it is much slower than using a physical machine.

The traditional x86 processor offers four instruction execution rings: Rings 0, 1, 2, and 3. The lower the ring number, the higher the privilege of the instruction being executed. The OS is responsible for managing the hardware and runs privileged instructions at Ring 0, while user-level applications run at Ring 3.

## 1.24 Virtualization of CPU, Memory, and I/O Devices

To support virtualization, processors such as the x86 employ a special running mode and instructions, known as **hardware-assisted virtualization**. In this way, the VMM and guest OS run in different modes, and all sensitive instructions of the guest OS and its applications are trapped in the VMM. Mode switching is completed by hardware, to save processor states efficiently. Intel and AMD each have proprietary technologies for hardware-assisted virtualization on the x86 architecture.

### 1.24.1 Hardware Support for Virtualization

Modern operating systems and processors permit multiple processes to run simultaneously. If there were no protection mechanism in a processor, all instructions from different

processes would access the hardware directly and could cause a system crash. Therefore, all processors have at least two modes:

- **User mode** — runs unprivileged instructions.
- **Supervisor mode** — runs privileged instructions, ensuring controlled access to critical hardware.

In a virtualized environment, correctness is harder to guarantee because there are more layers in the machine stack.

### 1.24.2 CPU Virtualization

A VM is a duplicate of an existing computer system in which the majority of VM instructions execute on the host processor in native mode. Thus, unprivileged instructions of VMs run directly on the host machine for higher efficiency. Other critical instructions must be handled carefully for correctness and stability. These critical instructions fall into three categories:

- **Privileged instructions** — execute in a privileged mode and are trapped if executed outside that mode.
- **Control-sensitive instructions** — attempt to change the configuration of resources used.
- **Behavior-sensitive instructions** — have different behaviors depending on the configuration of resources, including load and store operations over virtual memory.

A CPU architecture is **virtualizable** if it supports running the VM's privileged and unprivileged instructions in the CPU's user mode, while the VMM runs in supervisor mode. When privileged, control-sensitive, or behavior-sensitive instructions of a VM execute, they are trapped in the VMM, which acts as a unified mediator for hardware access across all VMs — ensuring correctness and stability for the whole system. RISC CPU architectures can be naturally virtualized because all control- and behavior-sensitive instructions are privileged instructions there.

#### Hardware-Assisted CPU Virtualization

This technique simplifies virtualization, since full or para-virtualization is otherwise complicated. Intel and AMD add an additional privilege level, called Ring -1, to x86 processors. As a result:

- The operating system can still run at Ring 0.
- The hypervisor runs at Ring -1.
- All privileged and sensitive instructions are trapped in the hypervisor *automatically*.

This removes the difficulty of implementing binary translation for full virtualization, and it also lets the operating system run inside VMs *without modification*.

### 1.24.3 Memory Virtualization

Virtual memory virtualization is similar to the virtual memory support provided by modern operating systems. In a traditional (non-virtualized) execution environment, the OS maintains mappings of virtual memory to machine memory using page tables — a **one-stage mapping** from virtual memory to machine memory. All modern x86 CPUs include a Memory Management Unit (MMU) and a Translation Lookaside Buffer (TLB) to optimize virtual memory performance.

In a virtual execution environment, memory virtualization involves sharing the physical system RAM and dynamically allocating it to the physical memory of the VMs. This requires a **two-stage mapping process**, maintained by the guest OS and the VMM respectively:

- Virtual memory → Physical memory (maintained by the guest OS)
- Physical memory → Machine memory (maintained by the VMM)

The guest OS continues to control the mapping of virtual addresses to the physical memory addresses of the VM, but it **cannot** directly access the actual machine memory. The VMM is responsible for mapping guest physical memory to the actual machine memory. MMU virtualization is transparent to the guest OS.

#### Architecture — Two-Level Memory Mapping Procedure

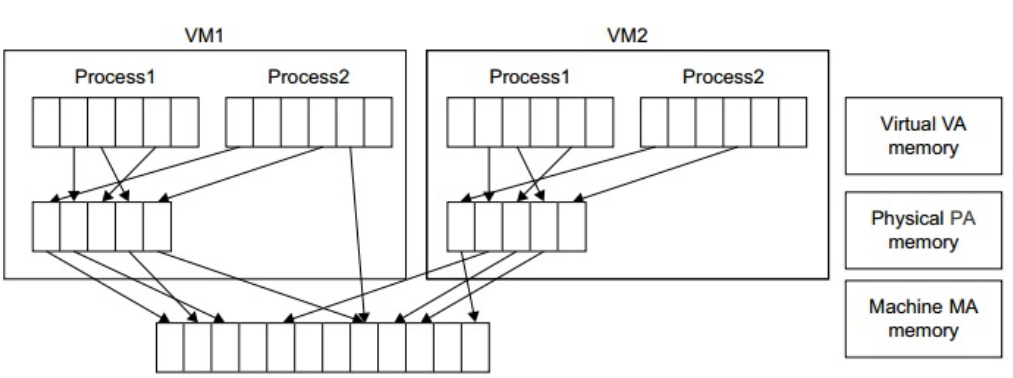



Figure 1.5: Two-level memory mapping procedure — per-process virtual (VA) memory in VM1 and VM2 is mapped to each VM's physical (PA) memory, which is in turn mapped to the host's real machine (MA) memory.

#### Practical Memory Management in a Hypervisor

Modern hypervisors add several techniques on top of the two-stage mapping to use physical RAM efficiently across many VMs:

- **Paging** — when memory blocks need to be freed for newer information, older, less-recently-used blocks (*pages*, typically 4 KB) are written to disk. The disk acts as an extension of physical memory; the file the pages are copied to is called the *page file*. Paging is far slower than working directly with memory.

- **Memory overcommitment** — allocating more virtual memory across VMs than the host physically has, on the assumption that VMs rarely use 100% of their allocated memory simultaneously.
- **Ballooning** — to reclaim physical memory from a VM under contention, a *balloon driver* inside the guest OS inflates, forcing the guest OS to flush its own least-recently-used pages to disk. Once the pages are flushed, the balloon deflates and the hypervisor reclaims that physical memory. This usually happens only when there is contention for memory.
- **Page sharing / copy-on-write** — memory pages with identical content (e.g., across VMs running the same OS) can be shared among multiple VMs on the same physical host, which is highly useful for read-only pages. If a VM needs to write to a page that has been marked as shared, the hypervisor creates a private copy for that VM — this is called **copy-on-write**. In practice, page sharing can save an additional 10% to over 40% of available physical memory, and it is especially valuable for Virtual Desktop Infrastructure (VDI), where many VMs run near-identical desktop images.
- **Swap space** — when a VM is first booted, the hypervisor reserves a disk-based swap space for storing memory pages if paging is required. If ballooning does not free enough memory, the hypervisor can swap the VM's memory pages out of physical memory entirely.

### 1.24.4 I/O Virtualization

I/O virtualization involves managing the routing of I/O requests between virtual devices and the shared physical hardware. There are three ways to implement I/O virtualization:

- Full device emulation
- Para-virtualization (split driver model)
- Direct I/O

#### Full Device Emulation

Full device emulation is the first approach to I/O virtualization. Generally, this approach emulates well-known, real-world devices. All the functions of a device or bus infrastructure — device enumeration, identification, interrupts, and DMA — are replicated in software. This software sits inside the VMM and acts as a virtual device. I/O access requests from the guest OS are trapped in the VMM, which interacts with the real I/O devices on the guest's behalf.

A single hardware device can be shared by multiple VMs running concurrently. However, software emulation runs much slower than the hardware it emulates.

#### Para-Virtualization I/O (Split Driver Model)

The para-virtualization method of I/O virtualization is typically used in Xen. It is also known as the **split driver model**, consisting of a *frontend driver* and a *backend driver*:

- The **frontend driver** runs in Domain U and manages the I/O requests of the guest OSes.
- The **backend driver** runs in Domain 0 and is responsible for managing the real I/O devices and multiplexing the I/O data of different VMs.

They interact with each other via a block of shared memory. Although para I/O virtualization achieves better device performance than full device emulation, it comes with a higher CPU overhead.

### Direct I/O Virtualization

Direct I/O virtualization lets the VM access devices directly, achieving close-to-native performance without high CPU costs. However, current direct I/O virtualization implementations mostly focus on networking for mainframes, and there are many challenges for commodity hardware devices — for example, when a physical device is reclaimed (required by workload migration) for later reassignment, it may have been left in an arbitrary state (e.g., mid-DMA to some arbitrary memory location) that can function incorrectly or even crash the whole system. Since software-based I/O virtualization requires a very high overhead of device emulation, hardware-assisted I/O virtualization is critical.

### The Guest OS I/O Stack (Inside the VM)

When an application inside a VM performs a disk or network operation, the request does **not** go straight to physical hardware. It first passes through the guest OS's own I/O stack:

- **Applications** — one or more apps issue reads/writes; the entry point of any I/O request.
- **Buffer cache** — recently used data is cached in memory to avoid slow disk access on repeat requests.
- **I/O scheduler** — orders and prioritizes queued requests to improve throughput and fairness.
- **Device driver** — the software layer the guest OS uses to talk to what it believes is real hardware.
- **Emulated disk device** — not a physical disk; the hypervisor presents a virtual disk that the guest OS treats as real.

After leaving the VM, the request passes through a second, independent I/O stack managed by the hypervisor on the host, before it ever reaches a real disk, NIC, or storage array:

- **Virtual-to-physical translation** — maps each VM's virtual disk blocks to real locations on physical storage.
- **Interpose / transform** — the hypervisor can intercept traffic to add logging, encryption, replication, or compression — transparently to the VM.
- **I/O scheduler** — fairly arbitrates I/O requests arriving from many competing VMs at once.

- **Device driver** — the real driver that communicates with actual physical hardware.
- **Local disk / NIC / NAS** — requests land on local physical disks, or travel over the network card to shared network-attached storage.

### Architecture — Hypervisor I/O Stack

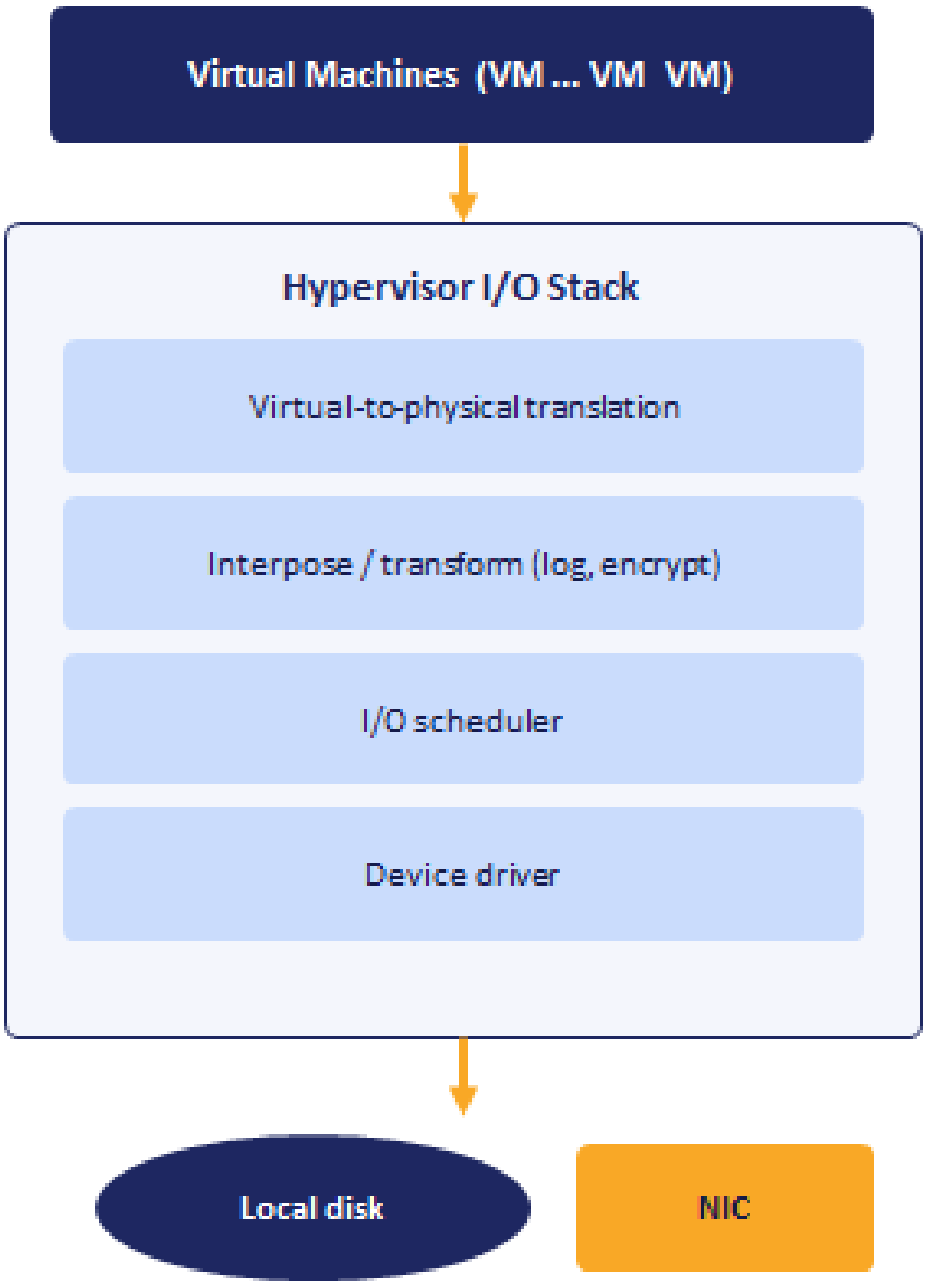



Figure 1.6: Hypervisor I/O stack at the host: Virtual Machines (VM1 ... VMn) → Virtual-to-physical translation → Interpose/transform (log, encrypt) → I/O scheduler → Device driver → Local disk / NIC / NAS.

### Guest Stack vs. Hypervisor Stack — Side-by-Side

tableheadfill

| | | |
|---|---|---|
| Scope | Handles one VM's own I/O | Handles I/O from all VMs on the host |
| Awareness of hardware | Believes it uses real hardware (emulated) | Talks directly to real physical hardware |
| Key extra step | Buffer cache for speed | Virtual-to-physical address translation |
| Special capability | None — standard OS behavior | Interpose/transform: log, encrypt, replicate |
| Final destination | Emulated disk device | Local disk, NIC, or remote NAS |

### Why Two Stacks? The Trade-off

- **The cost — overhead:** every I/O request now travels through two full stacks instead of one, adding extra processing steps (translation, scheduling twice), which makes virtualized I/O slightly slower than I/O on a plain, non-virtualized computer. *Example:* a database server running inside a VM typically sees somewhat higher disk latency than the same database running directly on hardware.
- **The benefit — new powers:** because the hypervisor sits in the middle of every I/O request, it can add valuable features transparently — without the guest OS or application ever knowing or needing to change. *Example:* the hypervisor can automatically encrypt every VM's disk writes, log activity for auditing, or move a VM's storage to a NAS — all invisibly to the VM.

### Follow One Request, Start to Finish

1. An application inside the VM issues an I/O request (e.g., saving a file).
2. The request enters the Guest OS I/O stack: the buffer cache is checked first, then the I/O scheduler queues it.
3. The guest's device driver sends the request to what it believes is a real disk — but it is actually emulated.
4. The hypervisor intercepts the request as it leaves the VM.
5. Virtual-to-physical translation converts the VM's virtual address into a real storage location.
6. The interpose/transform layer may encrypt, log, or otherwise modify the request — invisibly to the VM.
7. The hypervisor's I/O scheduler fits this request in fairly among all the other VMs requesting I/O at the same time.

8. The real device driver sends the request out to the local disk, or over the NIC to a NAS.

Steps 1–3 happen entirely inside the VM (the guest OS does not know it is virtualized); steps 4–8 happen at the host, invisibly to the guest.

### 1.24.5 Virtualization in Multi-Core Processors

Virtualizing a multi-core processor is relatively more complicated than virtualizing a unicore processor. Though multicore processors are claimed to have higher performance by integrating multiple processor cores in a single chip, multi-core virtualization raises new challenges for computer architects, compiler constructors, system designers, and application programmers. There are mainly two difficulties:

- Application programs must be parallelized to use all cores fully.
- Software must explicitly assign tasks to the cores — a very complex problem.

## 1.25 Desktop Virtualization

### 1.25.1 What Is Desktop Virtualization?

Desktop Virtualization runs a user's entire desktop environment — operating system, applications, and data — on a remote server instead of on the local device. The user's device only displays the desktop and sends back keyboard/mouse input.

**Everyday analogy:** it is like streaming a movie instead of downloading it. You do not need the movie file on your own device — you just see the picture. With Desktop Virtualization, you do not need the desktop's processing power on your device — you just see the screen.

#### Traditional Desktop vs. Virtualized Desktop

tableheadfill

| | | |
|---|---|---|
| Where it runs | Locally, on the user's own PC | On a server in the data center |
| Client device | Full PC required | A **thin client** — a lightweight, low-cost terminal with minimal local processing — is enough, since it only renders the remote screen |
| Software examples | — | VMware Horizon, Citrix Virtual Apps & Desktops, Microsoft Windows 365 |

A **thin client** is a lightweight terminal that does little local processing — it exists mainly to render the screen it receives from the data center and to send back keyboard/mouse

input.

### 1.25.2 Virtual Desktop Infrastructure (VDI)

VDI moves the desktop computing environment off the end-user's device and into centrally managed servers. Users see and interact with a full desktop, but all processing, storage, and applications actually run in the data center — the local device is essentially just a display and input terminal.

- Centralized management, security, and patching.
- Consistent desktop experience across devices.
- Data stays in the data center, not on endpoints.
- Access is controlled differently for local vs. remote users.
- A pool of servers hosts the virtual desktop sessions.

**Architecture — Virtual Desktop Infrastructure (VDI)**

### 1.25.3 VDI Access Flow: Local vs. Remote Users

1. The user's device requests a desktop session.
2. The data center's server pool delivers (streams back) the desktop session.
   - **Local users** connect directly to the data center over the trusted internal network.
   - **Remote users** must pass through a **firewall**, which authenticates and filters traffic before granting data-center access.

**Architecture — VDI Access Flow**

### 1.25.4 VDI in the Real World

| tableheadfill | |
|---|---|
| Call centers | Hundreds of agents need the same simple desktop; IT sets it up once in the data center instead of configuring hundreds of individual PCs. |
| Remote / hybrid work | Employees working from home connect securely to their office desktop without company data ever leaving the data center. |
| Schools & labs | Students get a fresh, identical desktop every session on shared, low-cost devices — no need for powerful individual computers. |
| Regulated industries | Banks and healthcare providers keep sensitive data centralized and secure, instead of scattered across employee laptops. |




Figure 1.7: VDI architecture — a pool of servers in the data center hosts virtual desktop sessions; end-user devices (thin clients / PCs / laptops) connect over the network and receive only the rendered desktop screen, sending back keyboard and mouse input.

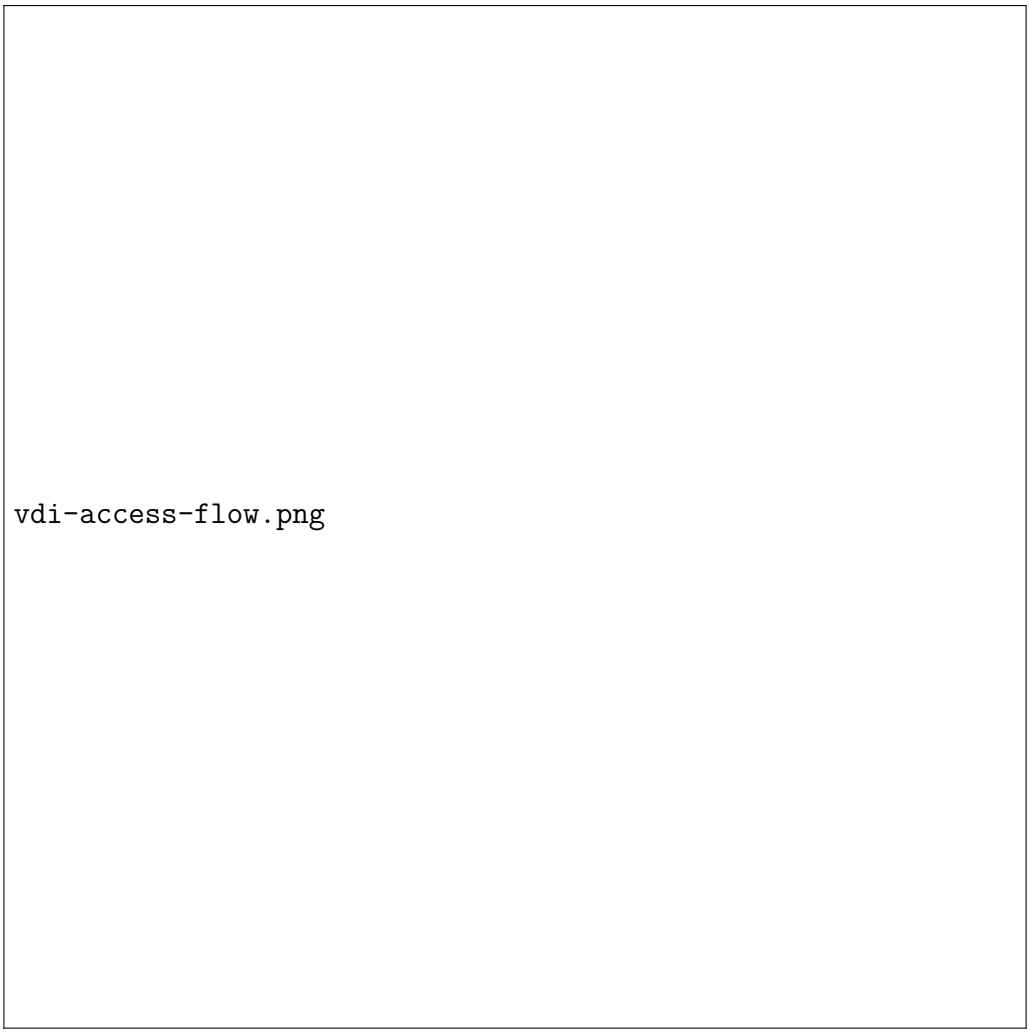



Figure 1.8: VDI access flow — a local user connects directly to the data center's server pool (SERVER, SERVER, ...); a remote user's request is routed through a firewall, which authenticates and filters traffic, before reaching the same server pool. Both flows: (1) request desktop session, (2) desktop session delivered/streamed back.

### 1.25.5 Popular Virtualization / VDI Software

tableheadfill

| | | |
|---|---|---|
| VMware ESXi / vSphere | Type 1 hypervisor | Industry-standard bare-metal hypervisor for enterprise data centers |
| Microsoft Hyper-V | Type 1 hypervisor | Built into Windows Server; widely used for server virtualization |
| Oracle VirtualBox | Type 2 hypervisor | Free, popular for running VMs on a personal desktop or laptop |
| VMware Workstation | Type 2 hypervisor | Commercial desktop virtualization for development and testing |
| KVM (Kernel-based VM) | Type 1 (Linux-integrated) | Open-source hypervisor built into the Linux kernel |
| Citrix / VMware Horizon | Desktop virtualization (VDI) | Deliver full virtual desktops to thin clients over the network |

# Quick Revision Summary

- **Virtualization structures:** hypervisor architecture (bare metal), para-virtualization (modified guest OS with hypercalls), and host-based virtualization (VMM on top of a host OS) are the three typical VM architectures. Xen splits guest domains into privileged **Domain 0** and unprivileged **Domain U**.
- **Full virtualization** = binary translation + direct execution; the guest OS is unmodified and unaware it is virtualized.
- **CPU virtualization** traps privileged, control-sensitive, and behavior-sensitive instructions into the VMM; hardware-assisted virtualization adds a Ring -1 for the hypervisor.
- **Memory virtualization** uses a two-stage mapping (virtual → physical → machine memory), plus techniques like ballooning, page sharing/copy-on-write, and memory overcommitment to use RAM efficiently across VMs.
- **I/O virtualization** has three approaches — full device emulation, para-virtualization (split frontend/backend driver model), and direct I/O — and every I/O request passes through two stacks: the guest OS I/O stack, then the hypervisor I/O stack.
- **Desktop Virtualization (VDI)** runs the entire desktop in the data center; thin clients just render the screen. Local users connect directly; remote users pass through a firewall.

## 1.26 Virtualization for Data Center Automation: The Big Picture

### 1.26.1 Start With a Simple Problem

Think about a huge data center run by a company like Amazon, Google or Microsoft. It holds **tens of thousands of servers**. Now imagine that every time a customer says, "I need a new server," a person has to walk into the room, pick a machine, install an operating system, connect cables, and set up the network.

That would be slow, expensive, and full of human mistakes. It simply cannot work at cloud scale.

> **The real-life picture:** An old-style telephone exchange needed a human operator to plug a wire in for every single call. Today, calls connect automatically in a fraction of a second. Virtualization does the same thing for servers — it lets *software*, instead of humans, create, move, and remove computers.

### 1.26.2 What Does "Automation" Mean Here?

Data center automation means that routine jobs happen by themselves, controlled by software:

- Creating a new server when a customer asks for one

- Deciding *which* physical machine should run it
- Adding more servers when many users arrive, and removing them when they leave
- Moving work away from a machine that is overloaded or about to fail
- Switching off machines that are not needed, to save electricity

**Core Concept**

Virtualization turns hardware into software-controlled building blocks. Once a server becomes a "virtual machine" (a file plus some settings), a program can create it, copy it, move it, or delete it — with no human touching the hardware.

### 1.26.3 Our Roadmap

This topic has two big parts. Each one answers a different question.

| Part | Question it answers | Main ideas |
|---|---|---|
| Virtual Clusters | How do we group virtual machines so they work together like one big system? | Physical vs. virtual cluster, architecture, dynamic provisioning, templates, images |
| Resource Management | How do we share CPU, memory, storage and network fairly and efficiently? | Scheduling, load balancing, consolidation, VM migration, fault tolerance |

## 1.27 Virtual Clusters

### 1.27.1 First, What Is a Cluster?

A **cluster** is a group of connected computers that work together to do a job, instead of depending on one very powerful computer.

**Everyday example — a restaurant kitchen.** One cook making 200 meals alone would be slow, and if that cook fell sick the restaurant would close. A team of cooks shares the work, and if one is absent the others carry on. A cluster is that team of cooks, but made of computers.

**University example.** A university may use three servers:

- Server 1 stores the **student database**
- Server 2 runs the **Learning Management System (LMS)**
- Server 3 runs the **examination system**

Together, these three servers form one cluster.




Figure 1.9: A cluster — a university uses three servers (student database, LMS, examination system) that work together.

#### 1.27.1.1 Why Do We Use Clusters?

| Benefit | Simple meaning |
|---|---|
| Higher performance | Many servers share the work, so jobs finish faster than on one machine. |
| Better availability | If one server is under maintenance, the others keep the service running. |
| Load sharing | Requests are spread across servers instead of overloading a single one. |
| Fault tolerance | If one server (node) fails, the whole system does not go down. |

| Benefit | Simple meaning |
|---|---|
| Scalability | When demand grows, you can simply add more servers. |

### 1.27.2 What Is a Physical Cluster?

A **physical cluster** is made of real, physical servers connected by a real network (cables and switches). Each server is a separate machine with its own dedicated CPU and RAM.

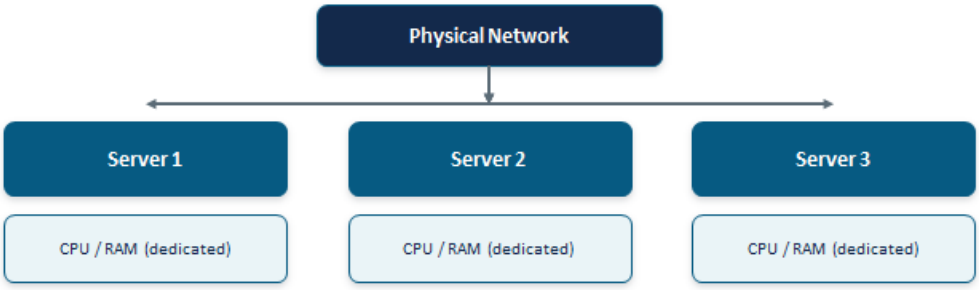



Figure 1.10: A physical cluster — three standalone servers, each with dedicated CPU and RAM, connected through a physical network.

**The problem:** What if you need one more server? You must buy it, deliver it, install it, and connect it. This is:

- **Expensive** — new hardware costs money
- **Time-consuming** — ordering and installing can take days or weeks
- **Less flexible** — you cannot "shrink" it easily when demand drops

**Real-life feeling:** It is like buying a new house every time a guest comes to stay, and then being stuck with an empty house after they leave.

### 1.27.3 Why Do We Need Virtual Clusters?

Now imagine that *one* physical server can run *several* virtual machines (VMs) at the same time. Each VM behaves like an independent computer, with its own operating system and applications.

| Stage | What happens |
|---|---|
| Before | 4 applications → 4 physical servers |
| After | 4 applications → 4 VMs → just 1 (or a few) physical servers |

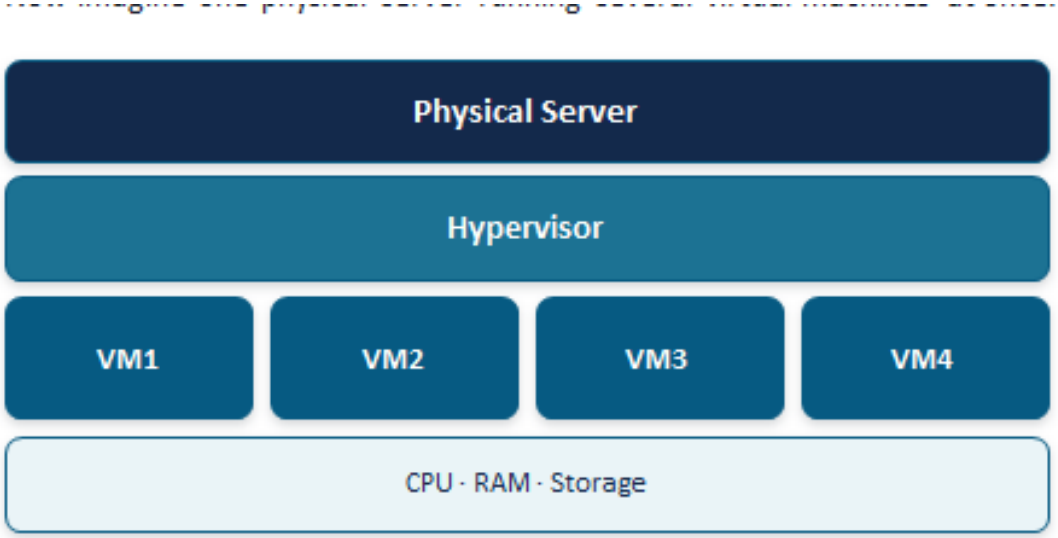



Figure 1.11: One physical server, with a hypervisor on top, running several independent VMs (before vs. after virtualization).

**Real-life example — a big flat divided into rooms.** Instead of building four small houses for four people, you build one large house and give each person a private room with their own lock and key. The house is the physical server; the rooms are the VMs. Everyone has privacy, but the building is shared and fully used.

So physical capacity is no longer tied one-to-one with applications. This gives better **resource utilization** and much more **flexibility**.

### 1.27.4 Definition of a Virtual Cluster

**Core Concept**

A **virtual cluster** is a collection of virtual machines, spread across one or more physical servers, and connected together through a virtual network.

Five features make a virtual cluster special:

1. **VMs may span hosts.** The member VMs can run on different physical machines, not only one.
2. **Virtual networking.** VMs talk to each other over a software-defined virtual network.
3. **Dynamic membership.** VMs can be added to or removed from the cluster whenever needed.
4. **OS heterogeneity.** Different operating systems (for example Linux and Windows) can run side by side on the same physical machine.
5. **Mobility.** A VM can be moved from one physical host to another while it keeps running.

### 1.27.5 Physical Cluster vs. Virtual Cluster

| Physical Cluster | Virtual Cluster |
|---|---|
| Uses physical servers | Uses virtual machines |
| Scaling needs new hardware | Scaling can be done by software |
| Less flexible | Highly flexible |
| Higher hardware cost | Better use of existing hardware |
| Migration is difficult | VM migration is possible |
| Provisioning takes a long time | Provisioning is dynamic and almost instant |
| Tied to the hardware | More hardware-independent |

**Small example:**

- *Physical cluster:* Server 1 → Application A, Server 2 → Application B, Server 3 → Application C
- *Virtual cluster:* Server 1 → VM1 + VM2, Server 2 → VM3 + VM4, Server 3 → VM5

In the virtual version, the same three servers now host five VMs, so nothing sits idle.

### 1.27.6 Virtual Cluster Architecture

Now let us see how all the parts fit together. A virtual cluster is built in layers, from the user at the top to the physical hardware at the bottom.

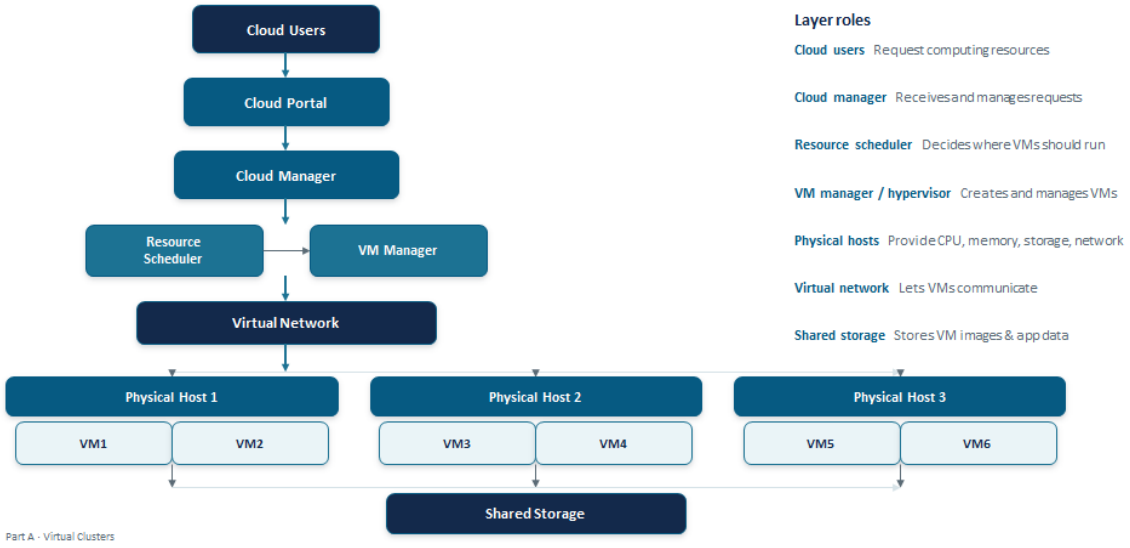



Figure 1.12: Virtual cluster architecture — from cloud users at the top down to physical hosts, VMs and shared storage.

| Layer | Role | Hotel comparison |
|---|---|---|
| Cloud users | Ask for computing resources | The guest asking for a room |
| Cloud portal | The website / interface where requests are made | The reception desk |
| Cloud manager | Receives and manages requests | The hotel manager |
| Resource scheduler | Decides where each VM should run | The person who assigns room numbers |
| VM manager / hypervisor | Creates and manages the VMs | Housekeeping preparing the room |
| Physical hosts | Provide CPU, memory, storage and network | The floors and rooms of the building |
| Virtual network | Lets VMs communicate | Corridors and lifts |
| Shared storage | Stores VM images and application data | The central storeroom |

### 1.27.7 How a Virtual Cluster Is Created (Dynamic Provisioning)

Suppose a researcher says:

> "I need a cluster of 5 Linux machines for a Big Data experiment."

The cloud handles this automatically, in these steps:

1. **User request** — the request arrives.
2. **Resource check** — the system checks how much capacity is free.
3. **Select physical hosts** — suitable machines are chosen.
4. **Create VMs** — the 5 virtual machines are made.
5. **Clone VM images** — the operating system and software are copied in.
6. **Configure the virtual network** — addresses and links are set up.
7. **Connect the VMs** — they are joined into one cluster.
8. **Cluster ready** — the user can start working.

**Core Concept**

This process is called **dynamic provisioning**. The number of nodes in the cluster can automatically increase or decrease depending on demand.

### 1.27.8 Real-Life Example: An Online Shopping Website

An online store normally needs only a few VMs. But demand is never constant.

| Situation | VMs running | What happens |
|---|---|---|
| Normal traffic | 3 VMs | Everyday shopping |
| Festival sale | 20 VMs | Thousands of shoppers arrive together |
| After the sale | 3 VMs | Extra VMs are removed |




Figure 1.13: Elasticity in action — an online shop grows from 3 to 20 VMs during a sale and shrinks back afterwards.

**Core Concept**

**Elasticity** means resources increase automatically when demand goes up, and decrease again when demand falls. A fixed physical cluster cannot do this without buying new hardware — a virtual cluster can grow and shrink freely.

**Real-life feeling:** A restaurant sets out extra tables on a festival day and folds them away the next morning. It does not build a bigger restaurant.

### 1.27.9 Template VMs: Make One, Copy Many

Imagine a cloud that must create thousands of VMs. Building each one from scratch would take forever. So the cloud first prepares one **template VM**.

- The template holds the common software: operating system, libraries, runtime environment, and common applications.
- Every new VM is a **clone** (copy) of that template.

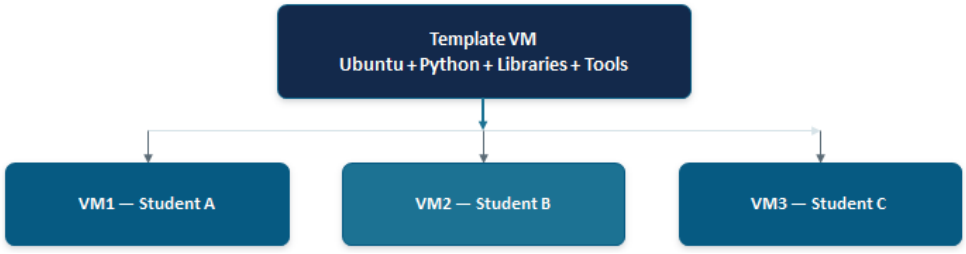



Figure 1.14: A template VM (Ubuntu + Python + libraries + tools) cloned into separate VMs for Student A, B and C.

**Real-life examples:**

- A teacher prepares one *master* question paper and photocopies it for every student.
- A school computer lab installs the software on one computer and then copies that setup to every other computer.

This reduces deployment time and makes image management much simpler.

### 1.27.10 VM Image Management

A **VM image** is the saved "package" of a virtual machine. It may contain the operating system, applications, libraries, settings, and user files.

Now think about the size of the job. If a cloud runs **10,000 VMs** and each image is **50 GB**, then:

$$10{,}000 \times 50 \text{ GB} = 500{,}000 \text{ GB} \approx 500 \text{ TB}$$

That is an enormous amount of storage. So clouds need efficient:

| Image task | Simple meaning |
|---|---|
| Storage | Keeping the images safely and compactly |
| Sharing | Letting many hosts use the same image |
| Replication | Keeping extra copies in case one is lost |
| Caching | Keeping popular images close by so they load fast |
| Distribution | Delivering images quickly to the hosts that need them |

**Real-life example:** Video apps like YouTube keep popular videos on servers near you so they start instantly. Clouds do the same with popular VM images.

Without good image management, dynamic provisioning and templates would be too slow and too costly at large scale.

## 1.28 Resource Management in the Cloud

### 1.28.1 What Is Resource Management?

**Core Concept**

**Resource management** is the process of allocating, monitoring, and optimizing cloud resources according to the needs of applications.

The main resources are **CPU** (thinking power), **RAM** (short-term memory), **storage** (disk space), and **network** (data connection).

The goal is to give every application what it needs, while avoiding:

- **Overloading** some servers

- **Underutilization** of other servers (sitting idle)
- **Performance degradation** (things becoming slow)
- **Unnecessary cost**

**Real-life example:** A traffic police officer does not build new roads; the officer directs vehicles so that no single road jams while others stay empty. Resource management does the same with computing work.

### 1.28.2 Why Is It Needed?

Suppose we have three servers.

| Server | CPU usage | Problem |
|---|---|---|
| Server A | 95% | Almost full — users feel slowness |
| Server B | 20% | Mostly idle — wasted capacity |
| Server C | 15% | Mostly idle — wasted capacity |

One server is overloaded while two sit nearly idle. This is **poor resource utilization**. Good resource management shifts some work from A to B and C, so that all three run at a comfortable level (for example, around 40–50% each) and performance stays smooth.

**Goal:** balance the workload + improve utilization + maintain performance.

### 1.28.3 Resource Management Architecture

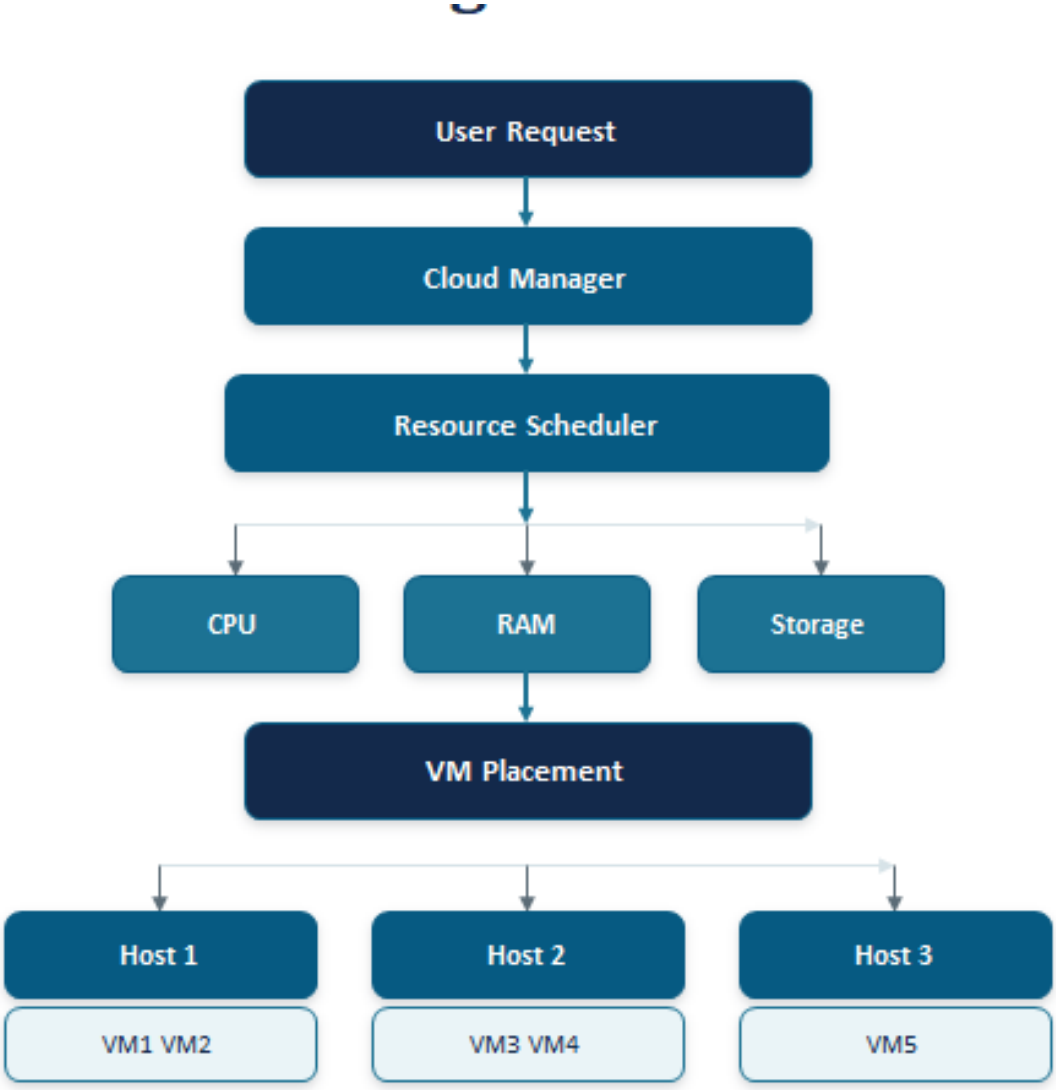



Figure 1.15: Resource management architecture — the Cloud Manager passes each request to the Resource Scheduler, which decides where VMs are placed.

Reading the flow from top to bottom:

1. A user request enters through the **Cloud Manager**.
2. The **Resource Scheduler** checks how much CPU, RAM and storage is available.
3. It decides **VM placement** — which physical host should run which VM.
4. The VMs are placed on hosts. The hosts are continuously monitored, so the cycle can repeat.

### 1.28.4 Main Functions of Resource Management

| Function | What it does | Everyday comparison |
|---|---|---|
| Resource allocation | Assigns CPU, memory, storage and network to workloads | Handing out desks in an office |
| Resource scheduling | Decides which VM gets resources, and where | Assigning rooms in a hotel |
| Monitoring | Tracks CPU, memory, network and storage usage | A fuel gauge and speedometer |
| Load balancing | Spreads work evenly across servers | Opening more billing counters in a supermarket |
| VM migration | Moves VMs between physical hosts | Shifting a shop to a new building |
| Server consolidation | Packs workloads onto fewer servers | Using fewer buses when few people travel |
| Fault management | Detects and recovers from failures | A backup generator during a power cut |

### 1.28.5 Worked Example 1: Resource Allocation

A physical host has **16 CPU cores** and **64 GB RAM**. Three VMs are placed on it:

| VM | CPU | RAM |
|---|---|---|
| VM1 | 4 cores | 16 GB |
| VM2 | 6 cores | 24 GB |
| VM3 | 2 cores | 8 GB |
| **Total used** | **12 cores** | **48 GB** |

$$\text{Remaining capacity} = (16 - 12) = 4 \text{ cores} \quad \text{and} \quad (64 - 48) = 16 \text{ GB}$$

The scheduler keeps a record of how much capacity is already committed. With 4 cores and 16 GB still free, it can accept one more suitably sized VM. If a request needs more than what remains, it must be **rejected or redirected** to another host, so the host does not get overcommitted.

### 1.28.6 Worked Example 2: Resource Scheduling

Scheduling answers one question: **Which physical machine should run this VM?**

A new VM needs **at least 4 CPU cores and at least 8 GB RAM**. The hosts look like this:

| Host | Free CPU | Free RAM | Fits? |
|---|---|---|---|
| H1 | 2 cores | 16 GB | No (CPU too low) |
| H2 | 8 cores | 4 GB | No (RAM too low) |
| H3 | 6 cores | 16 GB | **Yes** |

**Answer: Host 3.** It is the only host that satisfies both conditions at once.

**Lesson:** A scheduler must check *every* resource, not just one. It is like booking a hotel room that needs both a sea view *and* a double bed — a room with only one of the two is not good enough.

### 1.28.7 Load Balancing

**Problem:** Host 1 is at 95% CPU, while Host 2 is at 20% and Host 3 is at 25%. Host 1 is overloaded and slow, and the others have plenty of spare capacity.

**Solution:** Move some of Host 1's workload to Host 2.

| Host | Before | After | Change |
|---|---|---|---|
| Host 1 | 95% | 65% | −30% |
| Host 2 | 20% | 50% | +30% |
| Host 3 | 25% | 25% | no change |

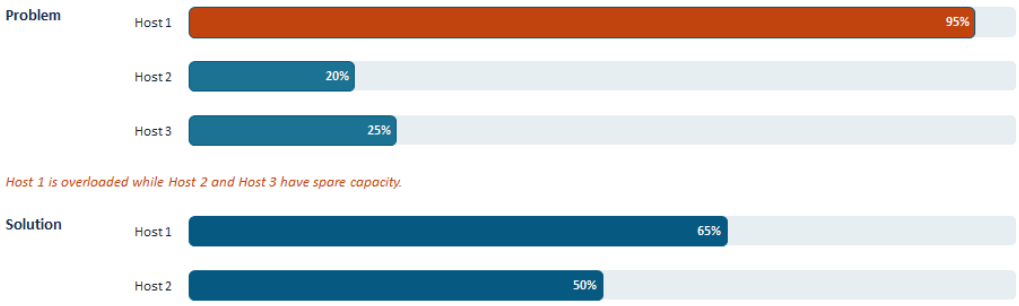



Figure 1.16: Load balancing — moving part of Host 1's work to Host 2 so that no host becomes a bottleneck.

**Core Concept**

**Load balancing** continuously redistributes work so that no single host becomes a bottleneck.

**Real-life example:** In a supermarket, if one billing counter has a long queue and the others are empty, the manager asks people to move to the free counters.

### 1.28.8 Server Consolidation

**Server consolidation** means running many workloads on *fewer* physical servers.

**Before:** four servers, each using only 20–30% of its CPU.

**After:** the four workloads are packed onto two servers, and the two empty servers are **switched off**.

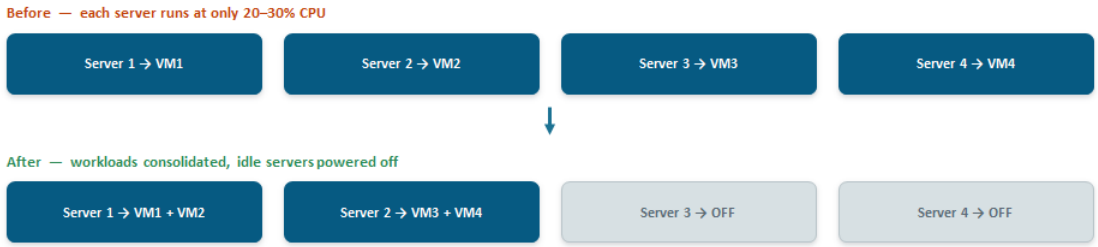



Figure 1.17: Server consolidation — four lightly used servers become two busy servers, and two are powered off.

**Advantages**

- Lower power consumption
- Lower cooling cost
- Better resource utilization
- Lower infrastructure cost

**Real-life example:** At night, a hostel keeps the lights on only in the rooms where students actually sleep and switches off the rest. (One caution: do not pack servers so tightly that a sudden rise in demand causes overload — keep some spare capacity.)

**Load balancing vs. consolidation:** They look like opposites, but they serve different goals. *Load balancing* spreads work out to protect **performance**. *Consolidation* packs work in to save **energy and cost**. A good cloud uses both, depending on the situation.

## 1.29 VM Migration

### 1.29.1 What Is VM Migration?

**Core Concept**

**VM migration** means moving a virtual machine from one physical host to another.

**Example:** Host A is running VM1 and VM2. Host B is empty. The cloud moves VM2 from Host A to Host B. Afterwards, VM1 stays on Host A and VM2 works on Host B.

### 1.29.2 Why Do We Migrate VMs?

| Reason | Explanation |
|---|---|
| Load balancing | Move a VM away from an overloaded server. |
| Server maintenance | Move VMs off a server before shutting it down for repairs or upgrades. |
| Fault avoidance | Move workloads away from a host that shows signs of trouble. |
| Energy efficiency | Consolidate workloads so that unused servers can be switched off. |
| Performance improvement | Move a VM to a host with better available resources. |

### 1.29.3 Live VM Migration

**Core Concept**

**Live migration** moves a *running* VM from one physical host to another with very little interruption to the service.

For example, a web server VM is busy serving users. The cloud moves it from Host A to Host B. The users keep using the website and usually do not even notice.

**Real-life example:** When you make a phone call while travelling in a train, your call passes from one mobile tower to the next. The call does not drop — you keep talking. Live migration works with the same spirit: the service continues while it moves.

Live migration is considered one of the major design issues in virtual clusters.

The VM's **execution state** — CPU registers, memory contents and device state — is sent across the network. The destination host then takes over and keeps the VM running.

### 1.29.4 Live Migration Steps

1. **Select the VM** that must be moved.
2. **Select the destination host.** It is chosen and reserved (preparation).
3. **Transfer memory.** Most memory pages are copied first, while the VM keeps running.
4. **Transfer changed memory.** Pages that changed during copying are sent again, in further rounds.
5. **Briefly suspend the VM.** It is paused for a very short time to copy the last small bit of changed state.
6. **Activate on the destination.** The VM resumes on the new host.

### 1.29.5 Worked Example of Live Migration

Host A is at **95% CPU** and runs three VMs:

| VM on Host A | CPU share |
|---|---|
| VM1 | 10% |
| VM2 | 30% |
| VM3 | 40% |

The three VMs add up to 80%. The remaining 15% is normally used by the hypervisor and other background work. The cloud manager decides to migrate **VM3** to Host B, which had spare capacity (assume it was at 25%).

| Host | Before | After |
|---|---|---|
| Host A | 95% | about 55% |
| Host B | 25% | about 65% |

**Small note:** If you count only the VMs that stay on Host A (VM1 + VM2 = 10% + 30% = 40%), you get 40%. Adding the 15% background usage gives about 55%. Either way, the two hosts are now much more balanced than 95% against 25%.

### 1.29.6 Migration Performance Factors

While a VM is being moved, three things have to be balanced:

| Factor | Question it asks |
|---|---|
| Downtime | How long is the service unavailable during the final "suspend and switch" step? |
| Network bandwidth | How much network capacity is used to send the memory and state? |
| Migration time | How long does the whole migration take, from start to finish? |

**Objective:** low downtime + low network usage + a reasonable migration time. These three pull against each other. For example, finishing very quickly uses lots of bandwidth; using little bandwidth makes the move slower. Real systems tune the memory transfer to find a good balance.

### 1.29.7 Memory Migration

A VM's running state lives mostly in **memory** (operating system, applications, running processes, cache and data). So moving a VM mainly means moving its memory.

**The challenge:** If a VM has 16 GB of memory, sending all of it needs a lot of network bandwidth — and the VM is *still running and changing its memory* while we copy.

**Real-life example:** Imagine photocopying a 500-page notebook while your friend is still writing in it. By the time you finish, some pages have changed.

#### 1.29.7.1 Iterative Memory Transfer (Pre-Copy)

The solution is to copy memory in **rounds**, without stopping the VM:

1. **Round 1:** copy most of the memory.
2. **Round 2:** copy only the memory that changed during round 1.
3. **Round 3:** copy only the memory that changed during round 2.
4. **Pause:** stop the VM for a very short time and copy the last tiny changes.
5. **Activate:** start the VM on the destination host.

**Illustration (numbers are only for understanding):**

| Round | Data copied | Changed meanwhile |
|---|---|---|
| Round 1 | 16 GB (almost everything) | about 2 GB |
| Round 2 | 2 GB | about 200 MB |
| Round 3 | 200 MB | about 20 MB |
| Final pause | 20 MB | — |

**Why does downtime become tiny?** Each round only copies what changed since the last round. Because each round is shorter, the VM has less time to change things, so the amount shrinks every time. In the end, so little is left that the VM needs to pause only for a moment.

### 1.29.8 File System Migration

The destination host must also be able to reach the VM's files. There are two common approaches.

| Approach | How it works | Trade-off |
|---|---|---|
| 1. Move the virtual disk | The whole virtual disk is copied to the destination before (or while) the VM moves. | Simple, but may move a lot of data. |
| 2. Shared / distributed file system | All hosts can reach the VM's files directly, so nothing needs to be copied. | Needs shared storage, but migration is much faster. |

**Real-life example:** Approach 1 is like carrying your files from one computer to another in a pen drive. Approach 2 is like keeping your files in Google Drive — every device can open them, so there is nothing to carry.

### 1.29.9 Network Migration

After moving, the VM must keep talking to other systems. Each VM carries a **virtual IP address** (its network address) and a **virtual MAC address** (its hardware-style ID). These move *with* the VM.

Because the VM keeps its network identity, existing connections can continue with very little disruption. The client still talks to "VM1" and does not care that VM1 now sits on Host B.

**Real-life example:** Mobile number portability. You can change your phone company or city but keep the same number, so everyone can still reach you.

**The three parts of a migration together:** *memory* (what the VM is thinking), *files* (what the VM has stored), and *network identity* (how others find the VM). All three must move correctly for the VM to feel "unmoved."

## 1.30 Dynamic Resource Allocation and Elasticity

### 1.30.1 Dynamic Resource Allocation

Cloud demand changes all the time, so the resources given to an application must change with it.

- Low demand → 2 VMs
- High demand → 10 VMs
- Very high demand → 20 VMs
- When demand decreases, the number scales back down: 20 → 10 → 2 VMs

Two useful words:

- **Scale out:** add more VMs when demand rises.
- **Scale in:** remove VMs when demand falls.

This is the direct mechanism behind the elasticity in the online-shopping example.

### 1.30.2 The Resource Management Decision Cycle

Resource management is not a one-time action. It is a **continuous feedback loop**:

1. **Monitor** — collect live CPU, memory and network data from every host.
2. **Analyze** — study the data to spot overload or underuse.
3. **Decide** — make a scheduling decision (allocate more? migrate a VM?).
4. **Act** — allocate or migrate resources. Then the cycle repeats.

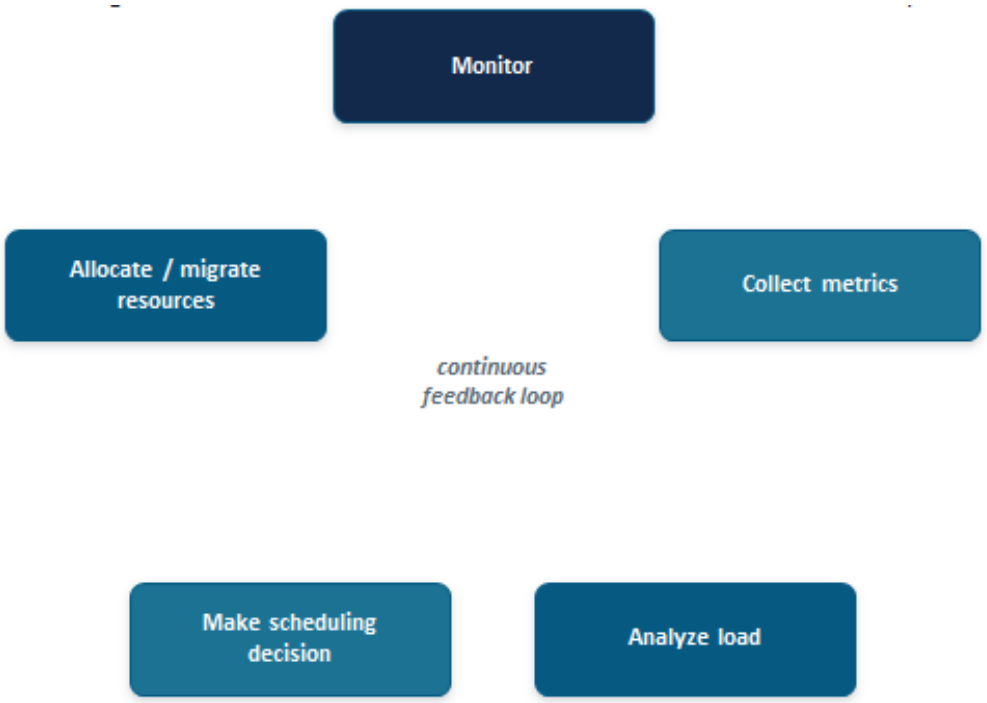



Figure 1.18: The resource management decision cycle — monitor, analyze, decide, act, and repeat.

**Real-life example:** A driver keeps checking the road, decides whether to speed up or slow down, presses the pedal, and then checks again — all the time.

### 1.30.3 Example: A University Cloud

A university cloud automatically changes the number of VMs to match the daily rhythm of college life:

| Time | Situation | VMs | Level |
|---|---|---|---|
| Morning | Normal workload | 10 VMs | Normal |
| 10 AM | Online classes | 20 VMs | Higher |
| Exam time | Peak demand | 50 VMs | Peak |
| Night | Minimal usage | 5 VMs | Low |

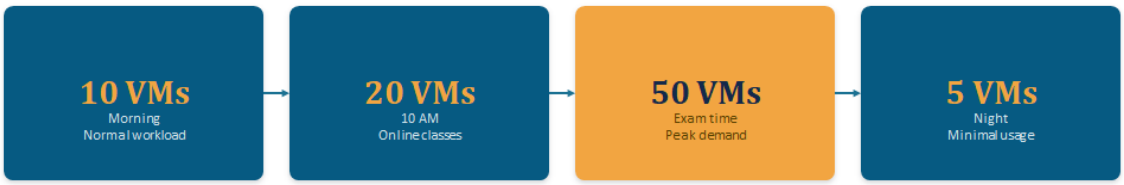



Figure 1.19: A university cloud scales up for online classes and exams, and scales down at night to save cost and energy.

### 1.30.4 Static vs. Dynamic Allocation

| Aspect | Static allocation | Dynamic allocation |
|---|---|---|
| Meaning | Resources are fixed in advance and do not change. | Resources change according to the real-time workload. |
| Example | VM1, VM2 and VM3 always get 4 CPUs each. | Low load → fewer resources; high load → more resources. |
| Good point | Simple. | Flexible and efficient. |
| Weak point | Less flexible, so resources are wasted or short. | Needs monitoring and smarter software. |
| Suitability | Steady, predictable work. | **Cloud environments**, where workload is rarely constant. |

**Real-life example:** Static allocation is like a fixed monthly electricity plan. Dynamic allocation is like a meter — you use, and pay for, what you need.

### 1.30.5 Summary of Resource Management Techniques

| Technique | Purpose |
|---|---|
| Resource allocation | Assign resources to workloads |
| Scheduling | Decide where workloads run |
| Load balancing | Distribute workloads evenly |
| VM migration | Move running workloads between hosts |
| Server consolidation | Reduce the number of active servers |
| Autoscaling | Increase or decrease resources with demand |
| Monitoring | Observe resource usage continuously |
| Fault tolerance | Handle failures gracefully |
| Energy management | Reduce power consumption |

### 1.30.6 Fault Tolerance

Virtualization makes recovery and relocation far more flexible than on plain physical hardware. Because a VM is just software (files and settings), it can be restarted on another machine.

**Without fault tolerance:** If Host A fails, VM1, VM2 and VM3 all fail with it.

**With fault tolerance:**

1. Host A fails.
2. The system **detects** the failure.
3. It **restarts or activates** the VMs elsewhere.
4. The VMs **resume on Host B**.

**Real-life example:** A hospital has a backup generator. When the main electricity fails, the generator starts, and the operation theatre keeps working.

## 1.31 Virtual Cluster Management Models

Who is in charge of managing the cluster? There are four common models:

| Model | Meaning |
|---|---|
| Guest-based management | The cluster manager runs *inside* the guest VMs. |
| Host-based management | The cluster manager runs at the physical host / infrastructure level. |
| Independent host and guest management | Separate management systems work at both levels. |
| Integrated management | Host and guest resources are managed together, as one system. |

### 1.31.1 Guest-Based vs. Host-Based Management

| Aspect | Guest-based | Host-based |
|---|---|---|
| Manager location | Runs inside the VM | Runs on the host / infrastructure |
| Visibility | Sees virtual resources | Sees physical resources |
| Best suited for | Virtual-cluster-aware software | Better infrastructure control |
| Involves | The guest OS | The hypervisor / host |

**Real-life comparison:** In a rented flat, a *guest-based* manager is like a tenant who organizes things only inside their own flat. A *host-based* manager is like the building manager who controls the lifts, water and electricity for the whole building.

The virtual cluster adds one extra layer (the VM and the hypervisor). This costs a little overhead, but in return you get much greater **flexibility, portability, and easier scaling**.

## 1.32 Advantages and Challenges of Virtual Clusters

**Advantages**

**Technical advantages**

- Better resource utilization
- Dynamic scaling
- VM isolation (one VM cannot easily disturb another)
- Easy provisioning
- VM migration
- Fault tolerance
- Load balancing
- Hardware independence

**Economic advantages**

- Lower hardware requirements
- Lower energy consumption
- Reduced maintenance
- Better use of existing infrastructure

**Disadvantages**

1. **VM overhead** — virtualization itself uses some CPU and memory.
2. **Migration overhead** — moving VMs uses network bandwidth.
3. **Resource contention** — many VMs compete for the same physical resources.
4. **VM image management** — huge numbers of images need efficient storage.
5. **Security** — a compromised VM may threaten the shared infrastructure.
6. **Scheduling complexity** — choosing the best host becomes harder at large scale.

**Real-life feeling for contention:** Flatmates sharing one kitchen and one bathroom. Usually it works well, but in the morning rush everyone waits.

## 1.33 Putting It All Together

### 1.33.1 Complete Resource Management Flow

Here is the whole story from the moment a user clicks a button:

1. **Users** submit requests through the **Cloud Portal**.
2. The **Cloud Manager** coordinates the request.
3. The **Resource Monitor** supplies live usage data, and the **Resource Scheduler** decides the placement.
4. **VM Provisioning** creates the VM through the **Hypervisor**.
5. The VM runs on a **physical host**, backed by **shared storage** and connected over the **virtual network**.

6. The Resource Monitor keeps watching, closing the loop (the decision cycle seen earlier).

### 1.33.2 End-to-End Example: A Machine-Learning Experiment

> "Create a virtual cluster with 4 VMs for a machine-learning experiment."

1. The cloud **receives** the request.
2. The scheduler **checks available resources**.
3. Four suitable VMs are **created**.
4. The VMs are **connected** through a virtual network.
5. VM images are **cloned from a template**.
6. The ML application is **deployed**.
7. The resource manager **monitors** CPU, RAM and network.
8. If Host 1 becomes overloaded, **VM2 live-migrates** to Host 3.
9. After completion, the 4 VMs are **terminated** and the resources are **released** for others.

Notice how this one example uses almost everything we studied: provisioning, templates, virtual networking, monitoring, live migration and release of resources.

### 1.33.3 Another Example: An E-Commerce Company

An e-commerce company runs **50 VMs across 5 physical servers**.

| Moment | CPU | What the resource manager does |
|---|---|---|
| Normal operation | 30% | Nothing special — just monitors |
| During a sale | 90% | **Scale out:** create more VMs, distribute the workload, monitor performance |
| After the sale | falls back | **Scale in:** remove unneeded VMs, release resources |

## 1.34 Virtualization for Data-Center Automation

### 1.34.1 What Is Data-Center Automation?

In recent years, data centers have grown very fast. Big technology companies such as Google, Amazon, Microsoft, IBM, HP, Apple and Yahoo! have spent billions of dollars building and automating them.

**Core Concept**

**Data-center automation** means that the huge pool of hardware, software and database resources inside a data center is handed out **dynamically** (by software, on demand) to millions of Internet users at the same time — with guaranteed **Quality of Service (QoS)** and at a reasonable cost.

**Real-life example:** Think of a huge railway reservation system. Millions of people book tickets at the same time, and nobody at the railway office manually assigns each seat. Software does it in seconds. Data-center automation is the same idea, but for servers, storage and software.

This automation is being pushed forward by two things: the growth of **virtualization products** and the spread of **cloud computing services**.

#### 1.34.1.1 Market Trend (A Little History)

A market report from IDC (published in 2007) estimated that virtualization business was worth about $1,044 million in 2006 and could grow to about $3.2 billion by 2011. Early on, most of the money came from *production consolidation* and *software development*. Later, attention moved toward:

- **High availability (HA)** — keeping services running even when something fails
- **Backup services** and disaster recovery
- **Workload balancing**
- **Utility computing** (pay for what you use) and more virtual clients
- **Mobility** (moving VMs around) and less *planned downtime* for maintenance

(These figures are old, but they show the direction: automation, service orientation, policy-based control and variable, pay-per-use costs.)

#### 1.34.1.2 The Four Topics We Will Study

| # | Topic | The question it answers |
|---|---|---|
| 1 | Server consolidation | How do we stop wasting half-empty servers? |
| 2 | Virtual storage management | How do we store thousands of VM disks efficiently? |
| 3 | Cloud OS for virtualized data centers | Which software manages the whole virtualized data center? |
| 4 | Trust management | How do we keep a shared, virtualized data center secure? |

### 1.34.2 Server Consolidation in Data Centers

#### 1.34.2.1 The Problem: Two Kinds of Workloads

A data center runs many different kinds of work at different times. Roughly, they fall into two groups:

| Workload type | Meaning | Example |
|---|---|---|
| Chatty workload | Interactive. It has busy bursts and quiet periods, depending on how many people are using it. | A web video service: crowded at night, almost empty in the daytime |
| Non-interactive workload | Once submitted, it keeps running on its own without a person guiding it. | A scientific simulation (high-performance computing), or a big report that runs overnight |

These workloads need very different amounts of resources at different moments. But to make sure the service never fails at its busiest time, the usual habit is **static allocation**: reserve enough resources for the *peak* demand and keep them reserved all the time.

**Result:** Most servers in a data center are **underutilized**. Hardware, floor space, electricity and management effort are all wasted.

**Real-life example:** A hotel that keeps every room lit and air-conditioned all day, just because on a few festival nights every room may be occupied.

#### 1.34.2.2 The Solution: Server Consolidation

**Core Concept**

**Server consolidation** improves the poor usage of hardware by **reducing the number of physical servers** that are needed.

There are several ways to consolidate (for example *centralized* and *physical* consolidation), but **virtualization-based consolidation is the most powerful**. Why?

- Older techniques work with a *whole machine* as the smallest unit. So resource management stays far from ideal.
- Server virtualization can hand out **smaller slices** than one whole physical machine (a few CPU cores, a few GB of memory).

**A small calculation.** Suppose a company has 10 servers, and each one is busy only 15% of the time.

$$\text{Total work} = 10 \times 15\% = 150\% \text{ of one server} = 1.5 \text{ servers' worth}$$

If we put this work into VMs on **3** servers, each server is about $1.5/3 = 50\%$ busy — a healthy level. So **7 servers** can be switched off, saving power, cooling and space. (In real life we also keep some spare room for sudden bursts.)

#### 1.34.2.3 Benefits of Virtualization-Based Consolidation

**Advantages**

1. **Better hardware utilization.** Many underused servers become a few well-used servers. Backup and disaster-recovery services also become easier.
2. **Faster, more agile provisioning.** The images of guest operating systems and their applications can simply be *cloned and reused*.
3. **Lower total cost of ownership.** New server purchases can be postponed, the data center needs less space, and maintenance, power, cooling and cabling costs go down.
4. **Better availability and business continuity.** If one guest OS crashes, the host OS and the other guests are not affected. Moving a VM to another server is easy because virtual servers do not depend on the details of the underlying hardware.

**Disadvantages**

- Using many VMs makes resource management **more complex**. The big challenge is how to improve utilization *and* still guarantee QoS.

#### 1.34.2.4 What Must Be Considered to Automate a Data Center?

- Resource **scheduling**
- **Architectural support** from the hardware and software
- **Power management**
- **Automatic (autonomic) resource management** — the system manages itself
- **Performance analytical models** — mathematical models that predict performance

In a virtualized data center, an **efficient, on-demand, fine-grained scheduler** is one of the most important tools for better utilization.

#### 1.34.2.5 Scheduling at Three Levels

Scheduling and reallocation can happen at three levels:

| Level | What is decided |
|---|---|
| VM level | How much CPU and memory each VM inside a server gets |
| Server level | How a physical server shares its resources among its VMs |
| Data-center level | Which servers are used, where new VMs are placed, and which servers can rest |

Ideally, all three levels would work together. But that is very complicated, so present techniques usually handle **only one level, or at most two**.

#### 1.34.2.6 Dynamic CPU Allocation and the Two-Level Controller

CPU is allocated dynamically using two kinds of information: how busy each VM is, and **application-level QoS metrics** (for example, "the web page must load within 2 seconds"). Some methods adjust both CPU and memory automatically as the workload of a hosted service changes.

To tame the complexity, one scheme uses a **two-level resource management system**:

- A **local controller** at the **VM level**
- A **global controller** at the **server level**

They talk to each other and together decide the resource allocation *autonomically* (without a human). **Real-life example:** In a college, each class teacher (local controller) knows what their class needs — projector, extra chairs, a bigger room. The principal's office (global controller) sees the whole college and decides who gets what. Neither can do the job well alone.

#### 1.34.2.7 Multicore Processors and Power Budgeting

Multicore processors (chip multiprocessors, CMPs) and virtualization can help each other, but multicore chips are still not used in the best possible way. For example, the **memory system** of a multicore chip can be improved by designing a **virtual hierarchy** for it. The goals are:

- Shorter memory access time
- Less **interference between VMs** (one noisy VM slowing its neighbours)
- Easier reassignment of VMs
- Safe sharing of data between VMs

Another idea is a **VM-aware power budgeting** scheme, where several managers work together to control power. Such policies must handle **heterogeneous** (mixed) hardware, and they always involve a **trade-off**: the more power you save, the more you risk data-center performance.

### 1.34.3 Virtual Storage Management

#### 1.34.3.1 What Does "Storage Virtualization" Mean Here?

The words "storage virtualization" existed long before system virtualization became popular, and the meaning has changed.

| When | Meaning |
|---|---|
| Older meaning | Combining and re-cutting physical disks, at a very coarse level, for use by *physical* machines. |

| When | Meaning |
|---|---|
| In system virtualization | The storage managed by the **VMM (hypervisor)** and by the **guest operating systems**. |

The data stored in a virtualized environment is of two kinds:

- **VM images** — special to virtual environments (the "packaged" VMs)
- **Application data** — everything else, the same as in a normal OS

#### 1.34.3.2 Encapsulation and Isolation

The two most important ideas of system virtualization are:

- **Encapsulation:** an operating system and its applications are wrapped up inside a VM as one neat package.
- **Isolation:** many VMs can share one physical machine, yet each is completely separated from the others.

**Real-life example:** Think of shipping containers. Each container packs its goods (encapsulation), and many containers can sit on the same ship without their contents mixing (isolation).

To support these ideas, CPUs, chipsets and system software have improved quickly. But **storage is lagging behind**, and so **storage has become the main bottleneck of VM deployment**.

#### 1.34.3.3 Why Is Storage So Difficult to Manage?

1. **The guest is being fooled.** A guest OS behaves as if it uses a real hard disk, but it is not allowed to touch the disk directly. The VMM sits in between.
2. **Many VMs fight for one disk.** When lots of VMs run on one machine, they compete for the same storage device.
3. **The VMM has a harder job** than a normal OS, because it must handle the requests of *all* guests.
4. **Storage tools are not nimble.** Actions like moving (remapping) a volume between hosts or taking a checkpoint of a disk are often clumsy, awkward, or sometimes not possible at all.
5. **Image flood.** Data centers may hold thousands of VMs, so the number of VM images becomes huge.

Researchers therefore try to make storage **easier to manage**, **faster**, and **smaller** (less space taken by VM images).

#### 1.34.3.4 Two Ideas: CAS and Parallax

- **Content Addressable Storage (CAS):** a technique that reduces the total size of VM images. Data is stored and found by *what it contains*, so content that is the

same in many images is kept only once instead of being saved again and again.

- **Parallax:** a distributed storage system designed specially for virtual environments.

**Real-life example for CAS:** A library keeps one copy of a textbook, and every student's reading list simply *points* to it, instead of storing 500 identical copies.

#### 1.34.3.5 Example: How Parallax Works

Ordinary storage management was designed without virtual environments in mind. Parallax takes a different path. Storage functions that used to sit inside expensive, high-end storage arrays and switches are **moved out into a group (federation) of storage VMs**. These storage VMs run on the *same physical hosts* as the VMs they serve.

Key points of the architecture:

- Each physical machine gets a special **storage appliance VM**. It works as a **block virtualization layer** between the client VMs and the physical storage device.
- The appliance VM gives **each VM its own virtual disk**, even though all of them share one large physical disk system.
- A **cluster-wide administrative domain** manages all the appliance VMs, so storage management is easy for the administrator.
- Features that once needed special hardware (like **snapshots**) can now be done in **software**, on top of ordinary, inexpensive network storage.
- Parallax itself runs as an ordinary **user-level application** inside the appliance VM. It works with both full virtualization and para-virtualization.
- The core object is the **Virtual Disk Image (VDI)**: a single-writer virtual disk that can be reached from *any* host of the Parallax cluster, no matter where it physically sits (location-transparent).
- It uses Xen's **block tap driver** (a *tapdisk* library) as one block-virtualization service for all client VMs on a host.
- The real hardware drivers live inside the appliance VM, so an administrator can **upgrade block device drivers while the cluster is running**.

**Real-life picture:** Imagine a big office building with a common warehouse. Instead of one central store-keeper far away, *every floor* has its own store-keeper (the storage appliance VM). Each employee (VM) is given a personal locker (virtual disk), while all lockers actually sit in the same warehouse. If the store-keeper's tools need an upgrade, it can be done on that floor without closing the whole office.

### 1.34.4 Cloud OS for Virtualized Data Centers

#### 1.34.4.1 What Is a Cloud OS?

To act as a cloud provider, a data center must first be virtualized. The software that does this is called a **virtual infrastructure (VI) manager** or **cloud OS**. It creates

VMs and groups them into **virtual clusters** that act as elastic resources. It is like an operating system, but for a whole data center instead of a single computer.

#### 1.34.4.2 Four Well-Known VI Managers

| Name | Type | Hypervisors used | Notable features |
|---|---|---|---|
| Nimbus | Open source | Xen, KVM | Virtual networking |
| Eucalyptus | Open source | Xen, KVM | Virtual networking; IaaS private clouds |
| OpenNebula | Open source | Xen, KVM | Adds dynamic resource provisioning and advance reservation |
| vSphere 4 | Proprietary (VMware) | ESX, ESXi | Virtual networking + **virtual storage** + data protection |

**Points to remember:** Nimbus, Eucalyptus and OpenNebula are free to use (open source). Only vSphere 4 is a commercial (proprietary) product. Only vSphere 4 supports virtual storage as well as virtual networking and data protection.

#### 1.34.4.3 Example 1: Eucalyptus

**What is it?** Eucalyptus is open-source software mainly meant for building **IaaS** clouds, especially **private clouds**. It mainly provides **virtual networking** and **VM management**; it does *not* provide virtual storage. A Eucalyptus cloud can talk to users over Ethernet or the Internet, and can also talk to other private or public clouds. It is short on security and some other features needed for general-purpose grid or cloud use.

**How is it built?** Each high-level part is a stand-alone **web service** with a clear, language-independent interface (described in a WSDL document). Secure communication between parts uses web-service security policies (WS-Security).

| Manager | What it does | Company comparison |
|---|---|---|
| Instance Manager | Runs on a host; controls the execution, inspection and termination of VM instances there | Shop-floor supervisor |
| Group Manager | Collects information, schedules VM execution on specific instance managers, and manages the virtual instance network | Regional manager |

| Manager | What it does | Company comparison |
|---|---|---|
| Cloud Manager | The entry point for users and administrators; asks for resource information, makes scheduling decisions and passes them to group managers | Head office |

Eucalyptus is designed to work like the **Amazon Web Services (AWS)** APIs, so it can interact with Amazon EC2. It also offers a storage API that imitates Amazon S3 for storing user data and VM images. It installs on Linux, is EC2-compatible (SOAP and Query) and S3-compatible (SOAP and REST), and can be used through a command-line interface or a web portal.

#### 1.34.4.4 Example 2: VMware vSphere 4 (a Commercial Cloud OS)

**What is it?** vSphere 4 is a commercial cloud OS released by VMware in April 2009. It extends earlier VMware products (VMware Workstation, ESX for server virtualization, and Virtual Infrastructure for clusters). It is mainly used to virtualize and manage data-center resources for **private clouds**. VMware described it as the first cloud OS that supports **availability, security and scalability**. Users and applications reach it through an interface layer called **vCenter**.

vSphere 4 has two groups of software:

**1. Infrastructure services (three packages).**

| Package | Built from | Simple meaning |
|---|---|---|
| vCompute | ESX, ESXi, DRS | Virtualizes the processors and memory |
| vStorage | VMFS, thin provisioning | Manages virtual disks. *Thin provisioning* gives a VM disk space only as it actually uses it. |
| vNetwork | Distributed switching | Virtual network switches that can span many hosts |

**2. Application services (three groups).**

| Group | Features | Simple meaning |
|---|---|---|
| Availability | VMotion, Storage VMotion, HA, Fault Tolerance, Data Recovery | *VMotion*: live migration of a VM. *Storage VMotion*: moving a VM's disk while it runs. *HA*: restart VMs on another host if a host fails. *Fault Tolerance*: a standby copy that takes over instantly. *Data Recovery*: backup. |
| Security | vShield Zones, VMsafe | *vShield Zones*: firewall-like protection between groups of VMs. *VMsafe*: lets security products inspect VMs safely. |
| Scalability | DRS, Hot Add | *DRS*: automatically balances load across hosts. *Hot Add*: add CPU or memory to a running VM without a restart. |

These infrastructure functions talk to the servers, disks and networks of the data center, and also to other external clouds.

**Note:** vSphere 4 is a 2009-era product; newer versions exist. The examples are used in textbooks because they show the *ideas* clearly: manage compute, storage and network together, and add availability, security and scalability on top.

## 1.34.5 Trust Management in Virtualized Data Centers

### 1.34.5.1 Why Does Virtualization Bring New Security Questions?

A VMM adds a new layer of software between the operating systems and the hardware. A VM **completely encapsulates** the state of the guest OS running inside it. That state can be **copied, shared over a network, and deleted like an ordinary file**. This is very handy — and it is also a security challenge.

- The VMM gives **secure isolation**, and every VM reaches the hardware only under the VMM's control. So the **VMM is the foundation of security** for a virtual system.
- Usually one VM acts as a **management VM**, which has special powers: create, suspend, resume and delete VMs.
- If a hacker breaks into the VMM or the management VM, **the whole system is in danger**.

**Real-life example:** A building's security depends on the master key and the security-control room. If a thief gets those, every flat is at risk.

#### 1.34.5.2 A Subtle Problem: Rolling Back a VM

Some security methods depend on a random number being **fresh** (never used before), for example when creating a session key.

Now suppose a VM is **rolled back** (restored to an earlier snapshot) at a moment *after* a random number was chosen but *before* it was used. The VM continues, and picks up the same "fresh" number **again**. Two different messages may then be encrypted with the same key stream (in a stream cipher), which could expose both messages. Also, reusing TCP initial sequence numbers can allow **TCP hijacking** attacks.

**Real-life example:** It is like a video game. You roll the dice, then reload an old save — and get exactly the same dice roll. In a game that is harmless; in security, a "one-time" secret that repeats is no longer secret.

#### 1.34.5.3 VM-Based Intrusion Detection

An **intrusion** is unauthorized access to a computer by a local or network user. An **intrusion detection system (IDS)** recognizes such access.

| Type | How it works | Weakness |
|---|---|---|
| HIDS (host-based) | Runs on the monitored system itself | If the system is attacked, the HIDS can be attacked too |
| NIDS (network-based) | Watches network traffic | Cannot see what really happens inside the host, so it can miss fake actions |

**Real-life comparison:** A HIDS is a guard standing *inside* the room — he sees everything, but a burglar can overpower him. A NIDS is a camera at the gate — it is safe, but it cannot see inside the rooms.

**The virtualization-based approach combines the good points of both.**

- The VMM **isolates** guest VMs on the same hardware. Even if one VM is broken into, the others are not affected (like a NIDS).
- The VMM also **monitors and audits** every request for hardware and system software, so fake actions can be caught (like a HIDS).

There are two ways to build a VM-based IDS:

1. The IDS runs as an **independent process in each VM**, or as a **high-privileged VM** on top of the VMM.
2. The IDS is **built into the VMM** and has the same privilege as the VMM for accessing hardware.

Garfinkel and Rosenblum proposed running the IDS as a **high-privileged VM on the VMM**.

- The IDS has a **policy engine** and a **policy module**. The policy framework watches events in different guest VMs through an OS interface library, and PTrace helps to follow the monitored host's activity against the security policy.
- **Logs matter.** It is hard to predict and stop every intrusion immediately, so *analysis after the attack* is very important. Most systems use logs for this, but a log can lose its credibility if an attacker can change it. A good design keeps the log service safe even when the guest OS has been invaded — which the VM-based design helps to do, because the IDS sits outside the guest.

**Real-life comparison:** A VM-based IDS is a security supervisor sitting in a control room outside all the rooms. Even if a burglar captures one room, the supervisor can still watch, record and raise the alarm.

#### 1.34.5.4 Honeypots and Honeynets

**Core Concept**

A **honeypot** is a deliberately weak, fake system that pretends to be a real one. Its purpose is to attract attackers, watch what they do, and learn from it. A group of honeypots is a **honeynet**.

- Attackers see a fake view of the system, so the **real system stays protected**.
- Their actions can be analyzed, which helps to build a **better, secure IDS**.
- A honeypot can be **physical** or **virtual**. In a virtual honeypot, a guest OS and its applications form a VM.
- The host OS and VMM must be protected so that an attack cannot escape from the honeypot VM.

**Real-life example:** Police sometimes leave a "bait" wallet or a decoy car to catch thieves and study how they work.

#### 1.34.5.5 Example: Trusted Zones for Virtual Clusters (EMC and VMware)

EMC and VMware together built **security middleware** for trust management in distributed systems and private clouds. The main idea is the **trusted zone**. Each tenant's virtual cluster (many applications and operating systems) is placed in its own separate virtual environment, forming a zone.

- The physical infrastructure at the bottom belongs to the **cloud provider**.
- Above it are the **virtual clusters of different tenants**, and at the top is the **public cloud** with global users.
- Security actions are applied at each level, between users, tenants and the provider: **antivirus, worm containment, intrusion detection, encryption and decryption**, and others.
- The main innovation is to **build trust zones between virtual clusters**.

- The end result is an **end-to-end view of security events and compliance** across all the virtual clusters used by different tenants.

**Real-life example:** A large apartment building. Each family has its own locked flat (a zone), while the building also has guards, CCTV and a visitor register that watch the whole building. A neighbour cannot walk into your flat, yet the building manager still has a complete view of what is happening.

### 1.34.6 Putting the Four Topics Together

| Topic | Problem | Main idea |
|---|---|---|
| Server consolidation | Servers are sized for peak, so most sit idle | Pack VMs on fewer servers; schedule at VM, server and data-center levels |
| Virtual storage management | Storage is the bottleneck of VM deployment; images flood the data center | CAS shrinks images; Parallax gives each VM a virtual disk through storage appliance VMs |
| Cloud OS | Someone must create and manage VMs and virtual clusters at scale | Nimbus, Eucalyptus, OpenNebula (open source) and vSphere 4 (commercial) |
| Trust management | Shared hardware and copyable VMs create new security risks | Protect the VMM; use VM-based IDS, honeypots and trusted zones |

### 1.34.7 Likely Exam Questions and Short Answers

| Marks | Question |
|---|---|
| 2 marks | What is data-center automation? What is server consolidation? Differentiate chatty and non-interactive workloads. What is a VDI in Parallax? What is a honeypot? |
| 5 marks | Explain the benefits of virtualization-based server consolidation. Explain virtual storage management and the Parallax architecture. Compare Nimbus, Eucalyptus, OpenNebula and vSphere 4. Explain VM-based intrusion detection. |
| 10 marks | Explain virtualization for data-center automation: server consolidation, virtual storage management, cloud OS, and trust management, with suitable diagrams and examples. |

**Model short answers:**

- **Data-center automation:** allocating hardware, software and data resources of a data center dynamically to millions of users at once, with guaranteed QoS and low cost.
- **Server consolidation:** reducing the number of physical servers by running their work as VMs on fewer machines, to improve utilization.
- **Chatty workload:** interactive work with bursts and quiet periods (e.g., a video service). **Non-interactive workload:** runs by itself after submission (e.g., high-performance computing).
- **VDI (Parallax):** a single-writer virtual disk that any host in the cluster can access, regardless of where it is physically stored.
- **Honeypot:** a deliberately weak fake system used to attract and study attackers.

### 1.34.8 Quick Revision Summary

- **Data-center automation** = dynamic, software-driven handing out of resources, with QoS and low cost; driven by virtualization and cloud services.
- **Two workload types:** chatty (bursty) and non-interactive (self-running). Static peak allocation wastes resources.
- **Consolidation benefits:** better utilization, faster provisioning by cloning, lower cost of ownership, better availability.
- **Scheduling levels:** VM, server, data center. Two-level control = local controller (VM) + global controller (server).
- **Storage:** data = VM images + application data. Storage is the bottleneck. **CAS** shrinks images; **Parallax** uses a storage appliance VM per host and single-writer VDIs.
- **Cloud OS:** Nimbus, Eucalyptus, OpenNebula (open source, Xen/KVM); vSphere 4 (proprietary, ESX/ESXi, adds virtual storage).
- **Eucalyptus:** Instance Manager → Group Manager → Cloud Manager; AWS-compatible; IaaS private clouds.
- **vSphere 4:** vCenter + infrastructure services (vCompute, vStorage, vNetwork) + application services (availability, security, scalability).
- **Trust:** VMM and management VM are critical; VM rollback can reuse "fresh" random numbers; VM-based IDS combines HIDS and NIDS advantages; honeypots lure attackers; trusted zones isolate tenants' virtual clusters.

### 1.34.9 The Big Question: Where Does Your Request Run?

"When you open an online shopping app, how does the cloud decide where your request should run?"

**The answer in plain words:** Your request reaches the cloud. The *resource manager* looks at how busy every machine is. The *scheduler* picks a physical host that has enough free CPU, memory and storage. A *virtual machine* on that host runs your request, and

the response comes back to your phone — all within a moment, with no human involved. If that host becomes too busy later, the cloud can quietly migrate the VM elsewhere.

## 1.35 Practice Section

### 1.35.1 Group Problem: Where Should the VM Go?

Three hosts, each with 16 cores. A new VM needs **4 CPU cores**.

| Host | CPU capacity | Current usage |
|---|---|---|
| H1 | 16 cores | 90% |
| H2 | 16 cores | 30% |
| H3 | 16 cores | 40% |

**Step 1 — Calculate the free cores.**

| Host | Used | Free | Can take 4 cores? |
|---|---|---|---|
| H1 | $0.90 \times 16 = 14.4$ | 1.6 | No |
| H2 | $0.30 \times 16 = 4.8$ | 11.2 | Yes |
| H3 | $0.40 \times 16 = 6.4$ | 9.6 | Yes |

**Answers to the classroom questions:**

1. **Which host is overloaded?** H1 (90% used, only 1.6 cores free).
2. **Where should the VM be placed?** On H2, which has the most free capacity (H3 would also work, but H2 leaves the system better balanced).
3. **Should a VM be migrated? Which one?** Yes. Live-migrate one VM from H1 to H3 (or H2). A VM using about 4–5 cores is a good choice, since it relieves H1 without overloading the destination.
4. **What is the objective of the migration?** Load balancing — to remove the overload on H1 and keep performance stable.
5. **What should be monitored?** CPU, RAM, network usage and storage on every host, continuously.

**Check:** After placing the new VM on H2 ($4.8 + 4 = 8.8$ cores, 55%) and moving a 4-core VM from H1 to H3 (H1 = 10.4 cores, 65%; H3 = 10.4 cores, 65%), all three hosts are at a healthy level. Remember: check *capacity and current usage* together before deciding.

### 1.35.2 Check Your Understanding

**Q1. Why don't we simply add a new physical server every time demand increases?**
It is expensive, slow, and leaves servers underused when demand later falls.

**Q2. What is the major advantage of VM migration?**
A workload can move between physical hosts without being manually reinstalled.

**Q3. Why is load balancing required?**
To stop some hosts from being overloaded while others sit underutilized.

### 1.35.3 Exam Preparation

| Marks | Likely questions |
|---|---|
| 2 marks | Define a virtual cluster. What is dynamic provisioning? What is resource management? What is server consolidation? What is a template VM? What is load balancing? Define live VM migration. |
| 5 marks | Explain physical and virtual clusters. Explain virtual cluster architecture. Explain live VM migration. Explain memory and file migration. Explain resource management techniques. |
| 10 marks | Explain virtual clusters and resource management in cloud computing with a suitable architecture. Discuss dynamic provisioning, VM migration, load balancing and server consolidation. |

**Model short answers (2 marks):**

- **Virtual cluster:** a collection of VMs spread over one or more physical servers and connected through a virtual network.
- **Dynamic provisioning:** automatically creating, configuring and connecting VMs on demand, so the cluster can grow or shrink.
- **Resource management:** allocating, monitoring and optimizing CPU, memory, storage and network according to application needs.
- **Server consolidation:** running many workloads on fewer physical servers so unused servers can be switched off.
- **Template VM:** one prepared VM with common software, from which new VMs are cloned.
- **Load balancing:** spreading workload evenly so no host becomes a bottleneck.
- **Live VM migration:** moving a running VM from one host to another with very little service interruption.

**Tip for the 10-mark answer:** Follow this order — definition → architecture diagram → dynamic provisioning → resource management functions → load balancing → server consolidation → live migration (memory, file, network) → advantages and challenges → a short real-life exampl